



**Институт интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Направление подготовки 09.03.04 Программная инженерия

Пояснительная записка

к учебно-исследовательской работе студента на тему:

**Разработка прототипа системы интеграции непрерывного
профилирования
в CI/CD-пайплайн для раннего обнаружения регрессий
производительности на языке Go**

Группа	<u>Б22-544</u>	
Студент	_____	<u>Портнов Л.М.</u>
	(подпись)	(ФИО)
Руководитель	_____	<u>Климов В.В.</u>
	(подпись)	(ФИО)
Научный консультант	_____	_____
	(подпись)	(ФИО)
Оценка руководителя (0–30 баллов)	Оценка консультанта (0–30 баллов)	
_____	_____	
Итоговая оценка _____ (0–100 баллов)	ECTS _____	

Комиссия

Председатель	_____	_____
	(подпись)	(ФИО)
	_____	_____
	_____	_____
	_____	_____



**Институт интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Задание на УИР

Студенту гр. Б22-544 Портнову Л.М.

ТЕМА УИР

**Разработка прототипа системы интеграции непрерывного
профилирования
в CI/CD-пайплайн для раннего обнаружения регрессий
производительности
на языке Go**

ЗАДАНИЕ

№ п/п	Содержание работы	Форма отчётности	Срок исполне- ния	Отметка о вы- полнении
				Дата, подпись
1.	Аналитическая часть			
1.1.	Обзор инструментов сбора и анализа данных о производительности в экосистеме Go	Подраздел ПЗ, список литературы		
1.2.	Сравнительный анализ подходов к интеграции профилирования и обнаружения регрессий в CI/CD	Подраздел ПЗ, сравнительная таблица подходов		
1.3.	Оформление расширенного содержания пояснительной записки (РСПЗ)	Текст РСПЗ		
2.	Теоретическая часть			
2.1.	Разработка модели процесса профилирования в CI/CD-пайплайне	Подраздел ПЗ, схема процесса		
2.2.	Формализация статистического критерия обнаружения регрессии производительности	Подраздел ПЗ, формулы критерия		

№ п/п	Содержание работы	Форма отчётности	Срок исполне- ния	Отметка о вы- полнении Дата, подпись
2.3.	Разработка концептуальной схемы интеграции двухуровневого профилирования в пайплайн	Подраздел ПЗ, схема артефактов		
3.	Инженерно-технологическая часть			
3.1.	Обоснование выбора модели жизненного цикла и инструментальных средств	Подраздел ПЗ		
3.2.	Проектирование архитектуры прототипа	Подраздел ПЗ, схема архитектуры		
3.3.	Описание состава и структуры основных компонентов системы	Подраздел ПЗ		
4.	Практическая часть			
4.1.	Разработка тестового Go-приложения с бенчмарками	Исходный код		
4.2.	Реализация критерия сравнения с эталоном (benchstat)	Исходный код		
4.3.	Реализация диагностического механизма сравнения rprof-профилей	Исходный код		
4.4.	Настройка CI/CD-пайплайна с профилированием (GitHub Actions)	Конфигурация CI, исходный код		
5.	Экспериментальная апробация			
5.1.	Подготовка сценариев внесения регрессий производительности	Исходный код (ветки regression/*)		
5.2.	Проверка корректности обнаружения регрессий на управляемых сценариях	Журналы экспериментов, таблицы результатов		
5.3.	Оценка накладных расходов профилирования в CI/CD	Журналы экспериментов, таблицы результатов		
5.4.	Сравнение и анализ результатов апробации	Сравнительный анализ, таблицы результатов		
6.	Оформление пояснительной записки (ПЗ) и иллюстративного материала для доклада	Текст ПЗ, презентация		

Дата выдачи задания:

Руководитель _____

Студент _____

Климов В.В.

Портнов Л.М.

Реферат

Пояснительная записка содержит 51 страницу текста, 5 рисунков, 6 таблиц, 2 приложения, список из 23 источников.

Ключевые слова: непрерывное профилирование, CI/CD, регрессия производительности, Go, pprof, benchstat, бенчмаркинг, статистическая значимость, критерий Уэлча, автоматизация тестирования.

Работа посвящена разработке и экспериментальной апробации прототипа системы, интегрирующей профилирование производительности Go-приложений в CI/CD-пайплайн для раннего обнаружения регрессий производительности — до попадания кода в основную ветку репозитория. Пояснительная записка состоит из пяти основных разделов: аналитического (обзор инструментов профилирования Go и подходов к их интеграции в CI/CD), теоретического (формализация критерия обнаружения регрессии и модели процесса профилирования), инженерно-технологического (проектирование архитектуры прототипа), практического (программная реализация прототипа и CI-пайплайна) и раздела экспериментальной апробации (проверка корректности обнаружения регрессий и оценка накладных расходов профилирования на четырёх управляемых сценариях). Работа завершается заключением с оценкой степени достижения поставленной цели и направлений дальнейших исследований.

Содержание

Реферат	2
Введение	5
1 Анализ инструментов профилирования Go и подходов к интеграции профилирования в CI/CD	7
1.1 Обзор инструментов сбора и анализа данных о производительности в экосистеме Go	7
1.2 Сравнительный анализ подходов к интеграции профилирования и обнаружения регрессий в CI/CD	8
1.3 Выводы	10
1.4 Цели и задачи УИР	11
2 Формализация критерия обнаружения регрессий и модель процесса профилирования в CI/CD	12
2.1 Модель процесса профилирования в CI/CD-пайплайне	12
2.2 Формализация статистического критерия обнаружения регрессии	13
2.3 Концептуальная схема интеграции двухуровневого профилирования в пайплайн	14
2.4 Выводы	15
3 Проектирование архитектуры прототипа системы профилирования	16
3.1 Обоснование выбора модели жизненного цикла и инструментальных средств	16
3.2 Архитектура прототипа	17
3.3 Состав и структура основных компонентов	17
3.4 Выводы	19
4 Программная реализация прототипа и модуля сравнения с эталоном	20
4.1 Разработка тестового Go-приложения как управляемой нагрузки	20
4.2 Реализация критерия сравнения с эталоном	20
4.3 Настройка CI/CD-пайплайна с профилированием	21
4.4 Тестирование прототипа	23
4.5 Выводы	23
5 Экспериментальная апробация прототипа на сценариях регрессий	25
5.1 Сценарии внесения регрессий и диагностика через rprof-профиль	25
5.2 Выявление отклонений и проверка корректности работы системы	26
5.3 Оценка накладных расходов профилирования в CI/CD	28
5.4 Выводы	29

Заключение	31
Список использованных источников	32
Приложение 1. Модульная спецификация и алгоритмическое описание	34
Приложение 2. Исходный код прототипа	37

Введение

Современные CI/CD-процессы к настоящему моменту достаточно надёжно контролируют *функциональную* корректность программного обеспечения: модульные и интеграционные тесты, статический анализ и линтеры являются стандартной, повсеместно принятой практикой и в большинстве промышленных пайплайнов блокируют слияние кода при обнаружении регрессии функциональности. *Производительность*, однако, в подавляющем большинстве пайплайнов остаётся вне поля автоматической проверки: регрессия времени выполнения, объёма потребляемой памяти или числа аллокаций типично обнаруживается постфактум — либо вручную при эпизодическом профилировании, либо только в продакшене, когда деградация уже сказалась на пользователях. Исторически интерес к профилированию производительности возникал прежде всего как реакция на инциденты в эксплуатации — от ранних систем трассировки вызовов до промышленных инфраструктур непрерывного профилирования дата-центров (Google-Wide Profiling [1]) и их современных открытых аналогов (Grafana Pyroscope [2], Parca [3]) — то есть решал задачу мониторинга уже выложенной системы, а не предотвращения попадания регрессии в неё. Актуальность настоящей работы состоит в переносе этой идеи на более ранний этап жизненного цикла — момент pull request, до слияния кода с основной веткой, — где стоимость исправления регрессии на порядки ниже, чем после развёртывания в продакшене.

Целью работы является разработка и экспериментальная апробация прототипа системы, интегрирующей профилирование производительности Go-приложений в CI/CD-пайплайн для автоматического обнаружения регрессий производительности на раннем этапе. Задачи, решение которых обеспечивает достижение этой цели, — анализ существующих инструментов и подходов, формализация критерия обнаружения регрессии, проектирование и программная реализация архитектуры прототипа, а также экспериментальная апробация на управляемых сценариях — подробно сформулированы по результатам аналитической части в разделе 1.

Теоретическая значимость работы заключается в формализации критерия обнаружения регрессии производительности, явно разделяющего статистическую значимость отклонения (критерий Манна — Уитни или Уэлча [4]) и его практическую величину, а также в разграничении понятия “непрерывное профилирование” на процессный смысл (встроенное в каждый прогон CI) и эксплуатационный смысл (постоянный мониторинг production-системы), что позволяет корректно позиционировать данную работу относительно существующих промышленных решений (раздел 1.2). Практическая значимость — в том, что предложенный прототип реализован как переносимый набор скриптов и CI-пайплайн, применимый к произвольному Go-проекту с бенчмарками без изменения кода слоя сравнения, что подтверждено архитектурным решением о разделении приложения и слоя сравнения (раздел 3).

Работа выполнена в рамках учебно-исследовательской работы студента и не является частью более широкого кафедрального или производственного проекта.

Пояснительная записка состоит из пяти основных разделов. Раздел 1 посвящён анализу инструментов профилирования и бенчмаркинга в экосистеме Go и сравнительному анализу существующих подходов к интеграции профилирования в CI/CD, по результатам которого формулируются цель и задачи работы. В разделе 2 формализуются модель процесса профилирования в CI/CD-пайплайне и статистический критерий обнаружения регрессии, сочетающий практическую и статистическую значимость отклонения. Раздел 3 описывает проектирование архитектуры прототипа: обоснование выбора модели жизненного цикла и инструментальных средств, состав и структуру основных компонентов системы. В разделе 4 излагается программная реализация прототипа — тестовое Go-приложение, реализация критерия сравнения с эталоном, настройка CI/CD-пайплайна и подходы к тестированию. Раздел 5 представляет результаты экспериментальной апробации прототипа на четырёх управляемых сценариях регрессии, включая оценку корректности обнаружения и накладных расходов профилирования. В заключении подводятся итоги работы и намечаются направления дальнейших исследований.

Раздел 1. Анализ инструментов профилирования Go и подходов к интеграции профилирования в CI/CD

1.1 Обзор инструментов сбора и анализа данных о производительности в экосистеме Go

Язык Go [5] изначально проектировался с расчётом на встроенную поддержку измерения производительности: инструментарий профилирования и бенчмаркинга поставляется вместе со стандартной библиотекой и инструментами go, а не выносится в сторонние пакеты, как это исторически происходило, например, в экосистеме Java (JMH подключается отдельно) или Node.js (clinic.js, 0x) [6].

Бенчмарки (testing.B). Пакет testing предоставляет тип testing.B, позволяющий описывать микробенчмарки функцией вида `func BenchmarkXxx(b *testing.B)`, где тело цикла выполняется `b.N` раз — количество итераций подбирается средой выполнения автоматически, пока не будет достигнута статистически достаточная длительность измерения. Команда `go test -bench=. -benchmem` выполняет такие функции и печатает три базовые метрики на итерацию: время (`ns/op`), объём выделенной памяти (`B/op`) и число аллокаций (`allocs/op`) [7]. Многократный запуск с флагом `-count=N` даёт выборку из N независимых измерений на каждый бенчмарк, что необходимо для последующего статистического сравнения, поскольку однократное измерение производительности крайне чувствительно к шуму окружения (частота планировщика ОС, состояние кэшей процессора, фоновая нагрузка CI-раннера) [8, 9].

benchstat. Утилита `golang.org/x/perf/cmd/benchstat` принимает один или два текстовых отчёта `go test -bench` и вычисляет для каждого бенчмарка медиану, доверительный интервал и, при сравнении двух наборов, относительное изменение метрики с p -value критерия Манна — Уитни (по умолчанию) или критерия Уэлча, если данные могут считаться нормально распределёнными [10, 4]. Именно benchstat лежит в основе сравнения ”текущий прогон vs. эталон” в данной работе (раздел 4) — однако сам по себе он не принимает решения ”регрессия / не регрессия”: он лишь предоставляет числа, интерпретация которых (выбор порога, уровня значимости) остаётся на усмотрение вызывающей стороны.

pprof (runtime/pprof, net/http/pprof). В отличие от агрегированных чисел бенчмарка, пакет `runtime/pprof` позволяет получить профиль — набор сэмплов стека вызовов, размеченных по времени (CPU-профиль, сэмплирование по таймеру) или по объёму выделенной памяти (heap/alloc-профиль, сэмплирование аллокатора). Профиль сохраняется в бинарном формате pprof (сжатый protobuf) и анализируется утилитой `go tool pprof`, которая умеет строить текстовые топ-листы (`-top`), графы вызовов (`-web`), построчные разборы горячих функций (`-list`) и, что особенно важно для настоящей работы, *разность двух профилей* через флаг `-diff_base`, вычитающую сэмплы одного профиля из другого функция за функцией [11, 12, 13]. Команда `go`

test умеет снимать такие профили прямо во время прогона бенчмарков флагами `-cpuprofile` и `-memprofile` — это единственный источник данных, показывающий, *какая именно функция* стала “тяжелее”, тогда как `benchstat` видит только агрегат по бенчмарку в целом.

go tool trace. Отдельный инструмент, визуализирующий исполнение горутин, работу планировщика и сборщика мусора во времени; полезен для диагностики задержек, вызванных конкуренцией за ресурсы (блокировки, GC-паузы), но не рассматривается подробно в данной работе, поскольку тестовое приложение (раздел 3) однопоточное по своей логике и не порождает конкурентного поведения, которое требовало бы такого анализа.

Промышленные системы непрерывного профилирования. Отдельный класс инструментов — системы, которые снимают `rprof`-совместимые (или `eBPF`-based) профили *непрерывно в продакшене*, а не разово в CI: инфраструктура Google-Wide Profiling, впервые описанная в [1], стала прообразом современных open-source решений — Grafana Pyroscope [2] и Parca (Polar Signals) [3], а также коммерческих сервисов (Google Cloud Profiler, Datadog Continuous Profiler). Эти системы решают принципиально другую задачу: они дают постоянный поток профилей с боевых инстансов для ретроспективного анализа (“что происходило в проде в момент инцидента”), а не блокируют слияние кода с обнаруженной регрессией. Практика эксплуатационного профилирования тесно связана с более широкой дисциплиной сопровождения production-систем (site reliability engineering), где постоянный сбор телеметрии, включая профили производительности, используется прежде всего для диагностики уже проявившихся инцидентов, а не для их предотвращения на этапе разработки кода [14]. Разграничение этих двух смыслов термина “непрерывное профилирование” — эксплуатационный (production continuous profiling) и процессный (профилирование при каждом прогоне CI до слияния) — формализуется в конце данного раздела и используется как рабочее определение на протяжении всей работы.

1.2 Сравнительный анализ подходов к интеграции профилирования и обнаружения регрессий в CI/CD

Обнаружение регрессий производительности до попадания кода в основную ветку концептуально относится к тому же классу задач, что и статический анализ или модульное тестирование в CI, но на практике встречается в пайплайнах заметно реже: по данным обзора существующих open-source решений (таблица 1.1) подавляющее большинство CI-конфигураций Go-проектов ограничивается `go vet`, `go test` и линтерами, не включая шаг сравнения производительности вовсе — `go test` сам по себе не проваливает сборку при деградации метрик, а лишь сообщает средние значения.

Из таблицы 1.1 видно, что существующие решения занимают две крайние точки: либо статистически строгое, но “слепое” к причине сравнение агрегатов (`benchstat`-класс инструментов), либо детальный, но не автоматизированный и не

Таблица 1.1 – Сравнение подходов к обнаружению регрессий производительности в CI/CD

Подход	Уровень данных	Статистическая строгость	Ограничение
Ручное профилирование разработчиком	rprof-профиль, разово	Отсутствует (субъективная оценка)	Не встроено в пайплайн, зависит от дисциплины разработчика
go test -bench без сравнения	Агрегат бенчмарка, разово	Отсутствует	Числа печатаются, но не с чем сравнивать автоматически
benchstat в CI (Bencher.dev, CodSpeed и др.)	Агрегат бенчмарка, серия	Есть (p -value, доверительный интервал)	Не показывает, какая функция деградировала — только итоговое число
JMH + CI-визуализация (экосистема Java)	Агрегат бенчмарка, серия	Есть (аналогично benchstat)	Не Go-специфично, приводится как референс зрелости подхода в другой экосистеме
Production continuous profiling	rprof/eBPF-профиль, непрерывно	Не применяется (не задача обнаружения регрессий)	Работает постфактум, после выкладки в прод, не блокирует PR
Прототип данной работы	Агрегат (benchstat) и rprof-профиль, при каждом PR	Есть (порог + p -value)	Синтетическая нагрузка бенчмарка может не отражать прод-трафик

привязанный к порогу профиль (pprof-класс инструментов и системы непрерывного профилирования). Ни один из рассмотренных инструментов не совмещает оба уровня данных в одном шаге CI-пайплайна. Это наблюдение определяет техническое решение, принятое в разделах 3–4: прототип запускает на каждый pull request как benchstat-сравнение (решение ”регрессия/ не регрессия”), так и параллельный съём pprof-профилей с их diff_base-сравнением (объяснение *какая функция стала причиной*), сохраняя оба артефакта.

Дополнительно был проведён обзор литературы по статистической методологии сравнения производительности программ. Работы Georges et al. [8] и Mytkowicz et al. [9] независимо показывают, что единичные измерения времени выполнения программ систематически вводят в заблуждение из-за детерминированных, но невидимых наблюдателю эффектов окружения (выравнивание кода и данных в памяти, порядок переменных окружения ОС), и что корректное сравнение обязано опираться на выборки и статистические критерии, а не на сравнение единственных чисел ”было/стало”. Это напрямую обосновывает использование в данной работе критерия Уэлча (либо непараметрического аналога) поверх выборки из -count повторов, а не сравнение единичных прогонов.

1.3 Выводы

1. Экосистема Go предоставляет развитый инструментарий сбора данных о производительности (testing.B, runtime/pprof, go tool pprof, golang.org/x/perf/cmd/benchstat), но не поставляет готового решения, принимающего бинарное решение ”регрессия обнаружена / не обнаружена” и блокирующего слияние кода на его основе.
2. Сравнительный анализ (раздел 1.2, таблица 1.1) показал, что существующие подходы делятся на статистически строгие, но агрегированные (benchstat-класс) и детальные, но не автоматизированные относительно порога (pprof-класс, включая промышленные системы непрерывного профилирования продакшен-систем — Pyroscope, Parca, Google-Wide Profiling); совмещения обоих уровней данных в одном шаге CI не обнаружено.
3. Показано (Georges et al. [8], Mytkowicz et al. [9]), что сравнение единичных измерений производительности статистически некорректно из-за скрытых эффектов окружения выполнения — критерий обнаружения регрессии обязан опираться на выборку повторных измерений и статистический критерий значимости, а не на разность двух отдельных чисел.
4. Определены два уровня метрик, необходимых для полноценного обнаружения и диагностики регрессии: агрегированные метрики бенчмарка (ns/op, B/op, allocs/op) и функциональные метрики профиля (flat/cumulative время CPU-профиля, alloc_space heap-профиля).

5. Зафиксировано рабочее разграничение термина ”непрерывное профилирование”: в данной работе оно означает профилирование, встроенное в каждый прогон CI-пайплайна (*процессный* смысл), в отличие от постоянного профилирования уже выложенного в продакшен приложения (*эксплуатационный* смысл, реализуемый системами вроде Pyroscope/Parca) — накладные расходы именно процессного профилирования количественно не изучены в рассмотренных источниках применительно к Go-бенчмаркам, что обосновывает необходимость эксперимента раздела 5.3.

1.4 Цели и задачи УИР

Цель работы — разработать и экспериментально апробировать прототип системы, интегрирующей два уровня данных о производительности Go-приложения — статистическое сравнение бенчмарков (benchstat) и pprof-диагностику на уровне функций — в CI/CD-пайплайн, чтобы регрессии производительности обнаруживались автоматически на этапе pull request, до попадания кода в основную ветку, а не постфактум в эксплуатации.

По результатам анализа, проведенного в разделе 1, были уточнены следующие задачи работы (аналитические задачи, выполнявшиеся на этапе РСПЗ, здесь заменены указанием, как их результат повлиял на решение остальных задач):

1. Результаты анализа (раздел 1.2) показали отсутствие решения, совмещающего агрегированное сравнение и pprof-диагностику, — это определило требование к архитектуре прототипа (раздел 3) включить оба механизма как параллельные, а не взаимоисключающие шаги пайплайна.
2. Формализовать критерий обнаружения регрессии производительности, опирающийся на статистическую значимость и практическую величину отклонения одновременно, а также модель процесса профилирования в CI/CD-пайплайне (раздел 2).
3. Спроектировать архитектуру прототипа: тестовое Go-приложение с воспроизводимой нагрузкой для бенчмарков, формат хранения эталонных данных (агрегированного и pprof-профилей), модуль сравнения и CI-пайплайн (раздел 3).
4. Реализовать прототип в виде исполняемых артефактов — тестовое приложение с бенчмарками, скрипты сравнения с эталоном (benchstat) и pprof-диагностики (diff_base), CI-пайплайн на GitHub Actions (раздел 4).
5. Провести экспериментальную апробацию на контролируемых сценариях регрессий: оценить корректность обнаружения (долю ложных срабатываний и пропусков) и количественно оценить накладные расходы, которые профилирование добавляет к длительности CI-джобы (раздел 5).

Раздел 2. Формализация критерия обнаружения регрессий и модель процесса профилирования в CI/CD

2.1 Модель процесса профилирования в CI/CD-пайплайне

Процесс обнаружения регрессии производительности рассматривается как конечный автомат, привязанный к событию открытия или обновления pull request (рис. 2.1). Ключевое проектное решение, вытекающее из вывода раздела 1.2, — после запуска бенчмарков процесс *разветвляется* на два параллельных, статистически независимых шага: сравнение агрегатов через benchstat (решающий шаг) и снятие pprof-профилей с их сравнением через diff_base (диагностический шаг, не влияющий на исход сборки, но прикладываемый к отчёту).

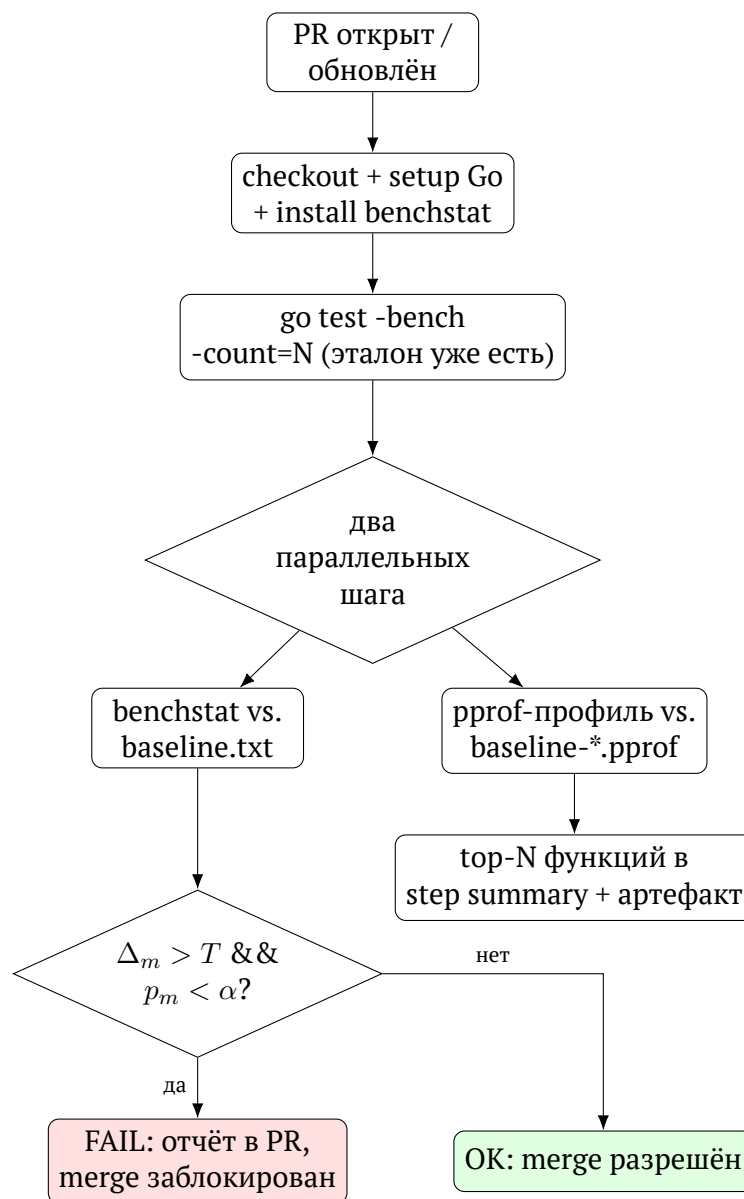


Рис. 2.1 – Модель процесса профилирования и обнаружения регрессии в CI/CD-пайплайне

Существенное свойство модели: диагностический шаг (правая ветвь, pprof-профиль) выполняется `always()` в терминах GitHub Actions — то есть независимо от исхода решающего шага, включая случай его падения. Это позволяет получить профиль-объяснение причины регрессии в том же прогоне CI, в котором она была обнаружена, не дожидаясь отдельного ручного профилирования разработчиком постфактум (что было отмечено как ограничение подхода ”ручное профилирование” в таблице 1.1).

2.2 Формализация статистического критерия обнаружения регрессии

Пусть на эталонном коммите (main) и на кандидатном коммите (PR) для каждого бенчмарка $b \in B$ и каждой метрики $m \in \{\text{ns/op}, \text{B/op}, \text{allocs/op}\}$ независимым запуском `go test -bench -count=n` получены выборки

$$X_{b,m} = \{x_1, \dots, x_n\}, \quad Y_{b,m} = \{y_1, \dots, y_k\},$$

где $X_{b,m}$ — измерения на эталоне, $Y_{b,m}$ — на кандидате (n и k обычно совпадают и равны параметру `-count`, но формально не обязаны). Все три метрики в данной работе относятся к классу ”меньше — лучше” (lower-is-better), поэтому регрессией считается *рост* значения.

`benchstat` оценивает центральную тенденцию каждой выборки медианой $\hat{\mu}(X_{b,m})$, $\hat{\mu}(Y_{b,m})$ и вычисляет относительное изменение

$$\Delta_{b,m} = \frac{\hat{\mu}(Y_{b,m}) - \hat{\mu}(X_{b,m})}{\hat{\mu}(X_{b,m})} \times 100\%. \quad (2.1)$$

Статистическая значимость отличия оценивается через p -value $p_{b,m}$ критерия Манна — Уитни (используется `benchstat` по умолчанию как непараметрический критерий, устойчивый к отклонению распределения задержек от нормального — задержки почти всегда правосторонне скошены из-за редких выбросов планирования ОС) либо критерия Уэлча [4] для приблизительно нормальных выборок. Регрессия по конкретной метрике конкретного бенчмарка объявляется тогда и только тогда, когда одновременно выполнены статистическая и практическая значимость отклонения:

$$\text{regression}(b, m) \iff (\Delta_{b,m} > T) \wedge (p_{b,m} < \alpha), \quad (2.2)$$

где T — порог относительного отклонения в процентах (параметр `THRESHOLD` в реализации, раздел 4), α — уровень значимости (параметр `ALPHA`). Условие $\Delta_{b,m} > T$ без учёта $p_{b,m}$ ловило бы шумовые колебания как ложные регрессии при малом T ; условие $p_{b,m} < \alpha$ без учёта $\Delta_{b,m}$ ловило бы статистически значимые, но практически пренебрежимо малые отклонения (например, 0,3% при большой выборке) — сочетание обоих условий необходимо именно для того, чтобы отделять шум от практически значимых регрессий; эта гипотеза проверяется экспериментально в разделе 5.2 перебором сетки значений T и α .

Итоговое решение по всему прогону — дизъюнкция по всем бенчмаркам и метрикам:

$$\text{FAIL} \iff \exists b \in B, m \in M : \text{regression}(b, m). \quad (2.3)$$

Диагностический критерий на уровне профиля. В отличие от критерия (2.2), сравнение pprof-профилей не участвует в принятии решения FAIL/OK, а используется описательно: для CPU-профиля каждая функция f характеризуется числом сэмплов `flat` (время, проведённое непосредственно в теле функции) и `sum` (суммарно с учётом вызываемых функций); `go tool pprof -diff_base` вычисляет

$$\delta_f = s_f^{\text{cur}} - s_f^{\text{base}} \quad (2.4)$$

раздельно для `flat` и `sum`, где s_f — число сэмплов, приходящихся на функцию f . Функции ранжируются по $|\delta_f|$, и `top- $N$` (в реализации $N = 10$) выводятся в отчёт. Аналогичное выражение применяется к heap-профилю с типом выборки `alloc_space` (кумулятивный объём выделенной памяти на функцию за время прогона).

Связь с алгоритмической сложностью. Отдельный частный случай регрессии, воспроизведённый в разделе 5 (сценарий `regression/quadratic-dedup`), — изменение класса временной сложности алгоритма. Если исходная реализация имеет сложность $T(n) = O(n)$ (проход по хеш-таблице), а изменённая — $T(n) = O(n^2)$ (вложенный линейный поиск, определение по [15]), то при фиксированном размере входа $n = \text{const}$, используемом в бенчмарке, оба варианта дают лишь *константный* множитель времени выполнения — сам факт квадратичной сложности не проявляется в единственном измерении при фиксированном n . Изменение класса сложности обнаруживается критерием (2.2) косвенно, через рост $\Delta_{b,m}$ при неизменном n , но не отличается формально от ”обычного” замедления в k раз без изменения сложности — для строгого различения потребовался бы бенчмарк с варьируемым n и оценка показателя роста, что выходит за рамки данной работы и отмечено как направление дальнейшего исследования в заключении.

2.3 Концептуальная схема интеграции двухуровневого профилирования в пайплайн

Формализованный процесс (рис. 2.1) и критерий (2.2) определяют требования к артефактам, которые должна хранить и обновлять система (рис. 2.2): эталонный агрегированный отчёт (`baseline.txt`), эталонные pprof-профили (`baseline-cpu.pprof`, `baseline-mem.pprof`) и параметры критерия (T , α , `-count`), являющиеся внешними по отношению к коду параметрами запуска, а не константами внутри приложения — это позволяет проводить эксперименты раздела 5 (перебор сетки $T \times \alpha$) без изменения кода прототипа.

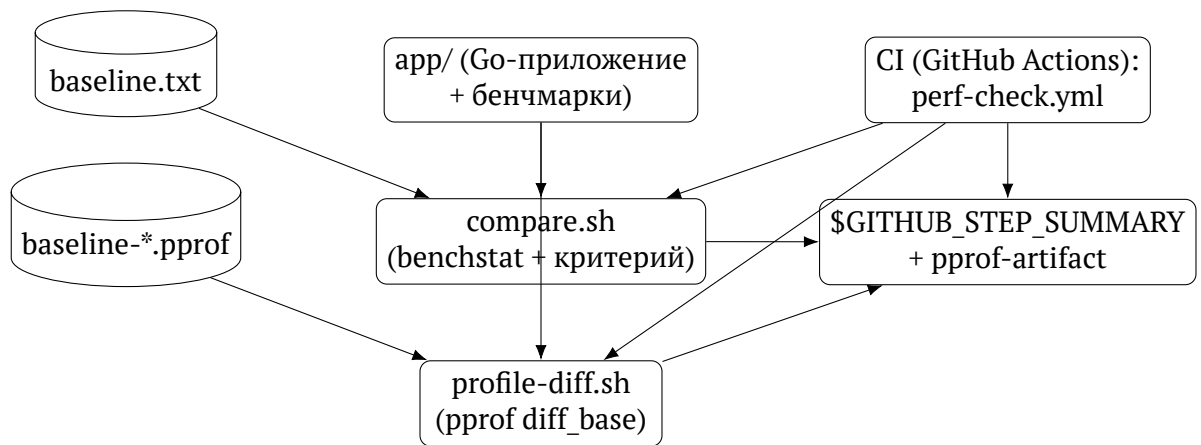


Рис. 2.2 – Концептуальная схема артефактов и компонентов интеграции профилирования в CI/CD

2.4 Выводы

1. Разработана модель процесса профилирования в CI/CD-пайплайне (рис. 2.1) в виде схемы с ветвлением на решающий шаг (benchstat-сравнение) и диагностический шаг (pprof-diff_base), выполняемые параллельно и независимо в рамках одного прогона CI.
2. Формализован критерий обнаружения регрессии (2.2), сочетающий практическую значимость отклонения $\Delta_{b,m}$ (выражение (2.1)) и статистическую значимость $p_{b,m}$; итоговое решение по сборке определено как дизъюнкция по всем бенчмаркам и метрикам (выражение (2.3)).
3. Формализован диагностический критерий на уровне отдельной функции профиля (2.4), не влияющий на решение FAIL/OK, но предоставляющий объяснение причины регрессии без отдельного ручного профилирования.
4. Показано (со ссылкой на определение алгоритмической сложности по [15]), что изменение класса временной сложности алгоритма при фиксированном размере входа n бенчмарка формально неотлично от обычного замедления в константное число раз — это ограничение модели явно зафиксировано и учтено при интерпретации сценария regression/quadratic-dedup в разделе 5.
5. Спроектирована концептуальная схема артефактов (рис. 2.2): параметры критерия (T , α , -count) и эталонные данные (текстовый отчёт и бинарные pprof-профили) вынесены за пределы кода приложения, что позволяет проводить экспериментальную апробацию (раздел 5) без изменения исходного кода прототипа.

Раздел 3. Проектирование архитектуры прототипа системы профилирования

3.1 Обоснование выбора модели жизненного цикла и инструментальных средств

Работа носит исследовательско-прикладной характер: на момент постановки задачи (раздел 1) не было известно заранее, какой именно набор параметров критерия (2.2) (T , α , -count) окажется работоспособным на выбранной тестовой нагрузке — ответ на этот вопрос сам является результатом эксперимента (раздел 5). По этой причине для разработки прототипа выбрана *эволюционная (итеративная) модель жизненного цикла* вместо каскадной: каждая итерация добавляла минимально достаточный слой функциональности (сначала тестовое приложение и бенчмарки, затем статистическое сравнение, затем pprof-диагностика, затем эксперименты) с немедленной проверкой работоспособности перед переходом к следующему слою. Такой подход соответствует рекомендациям ГОСТ Р ИСО/МЭК 12207 [16] в части организации процессов жизненного цикла программных средств для случая, когда требования уточняются по ходу разработки; в отличие от стадийной модели ГОСТ 34.601-90 [17] (ориентированной на последовательные стадии “техническое задание — эскизный проект — технический проект — рабочая документация”), выбранная модель не предполагает фиксации всех проектных решений до начала реализации. Такой подход согласуется с принципом коротких обратных связей, описанным в контексте практик непрерывной интеграции у Fowler [18] и Humble & Farley [19].

Выбор инструментальных средств обоснован следующими требованиями:

- **Язык реализации — Go 1.22 [20].** Определён темой работы; кроме того, Go — единственный из мейнстримных языков, поставляющий статистический сравнитель бенчмарков (benchstat) и профилировщик (pprof) как часть официальной экосистемы, а не сторонний пакет, что снижает поверхность интеграции.
- **CI-платформа — GitHub Actions.** Выбрана как наиболее распространённая платформа для open-source Go-проектов на момент разработки, с нативной поддержкой шага `$GITHUB_STEP_SUMMARY` для публикации markdown-отчёта прямо в интерфейсе pull request и встроенным механизмом артефактов (actions/upload-artifact) для сохранения бинарных pprof-профилей [21].
- **Формат хранения эталона.** Агрегированный эталон — текстовый вывод `go test -bench` (человекочитаемый, диффится в git); профильный эталон — бинарные .pprof-файлы (protobuf/gzip), хранящиеся в репозитории как обычные версионизируемые бинарные артефакты, без внешнего объектного хранилища — оправдано малым размером тестового приложения (профили десятки — сотни килобайт).

- **Язык скриптов интеграции — bash.** Выбран вместо, например, Go-программы-оркестратора, поскольку вся логика сводится к последовательному вызову внешних инструментов (`go test`, `benchstat`, `go tool pprof`) и разбору их текстового вывода — задача, для которой оболочка является более простым и прозрачным решением, чем компилируемая программа с дополнительной точкой сборки. Выбор именно `bash`, а не POSIX-совместимого `sh`, осознан: скрипты используют массивы и конструкцию `[[ ]]` (см. `scripts/lib.sh`), которые не входят в POSIX и недоступны в классическом `sh` — это сознательный отказ от переносимости на не-`bash`-окружения ради читаемости, поскольку целевая среда выполнения (раннер `ubuntu-latest` и локальная машина разработчика) гарантированно предоставляет `bash`.

Требования к прототипу как программной системе сформулированы следующим образом. **Функциональные:** (1) выполнить набор бенчмарков заданное число раз; (2) статистически сравнить результат с эталоном по критерию (2.2) и вернуть код завершения, пригодный для блокировки слияния; (3) снять CPU- и heap-профили того же прогона и сравнить их с эталонными профилями на уровне отдельных функций; (4) опубликовать оба результата в человекочитаемом виде в теле `pull request`. **Системные:** воспроизводимость (одинаковые входные данные и параметры дают одинаковое решение с точностью до статистического шума запуска), независимость от внешнего состояния (отсутствие сетевых зависимостей во время сравнения, кроме однократной установки `benchstat`), совместимость с `ubuntu-latest`-раннером GitHub Actions без дополнительных прав. **Требования к пользовательскому интерфейсу:** система не имеет графического интерфейса — её "интерфейсом" является (а) консольный вывод скриптов при локальном запуске и (б) `markdown`-отчёт в `$GITHUB_STEP_SUMMARY` при запуске в CI; оба должны оставаться читаемыми без дополнительной постобработки (табличный `markdown`, без ANSI-последовательностей).

3.2 Архитектура прототипа

Архитектура прототипа (рис. 3.1) состоит из четырёх слоёв: приложения под нагрузкой (`app/`), эталонных данных (`benchmarks/`), слоя сравнения (`scripts/`) и слоя оркестрации CI (`.github/workflows/`). Слой сравнения намеренно отделён от приложения: ни один из скриптов сравнения не импортирует пакет `app` и не содержит специфичной для него логики — единственная связь между слоями осуществляется через текстовый/бинарный формат вывода `go test`, что позволяет использовать те же скрипты для любого Go-пакета с бенчмарками без модификации (общность подтверждена тем, что `scripts/compare.sh` параметризован переменной окружения `PKG`).

3.3 Состав и структура основных компонентов

Приложение под нагрузкой (`app/`). Не является предметом исследования само по себе — это управляемая, воспроизводимая нагрузка, на которой демонстриру-

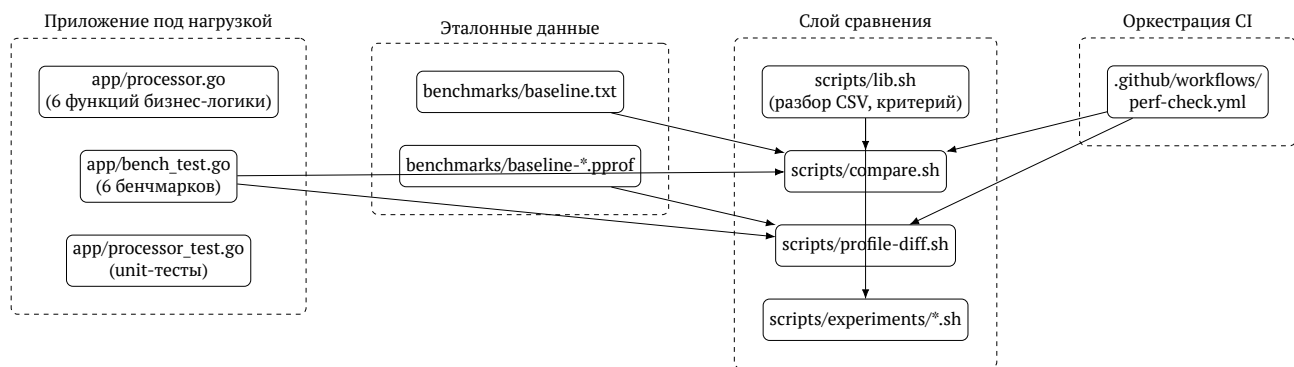


Рис. 3.1 – Архитектура прототипа системы непрерывного профилирования

ется работа системы профилирования. Состоит из шести функций обработки записей (ParseRecords, FilterActive, FindByID, Aggregate, FormatNames, Deduplicate), каждая из которых сознательно выбрана так, чтобы охватить разный характер нагрузки: разбор JSON (I/O-подобная, много аллокаций), линейная фильтрация и поиск (CPU-bound без аллокаций), агрегация в map (смешанная), дедупликация (структура данных, чувствительная к алгоритмической сложности — используется как раз для сценария regression/quadratic-dedup в разделе 5).

Эталонные данные (benchmarks/). Два независимых представления одного и того же эталонного прогона: baseline.txt — агрегат для benchstat, и пара baseline-cpu.pprof / baseline-mem.pprof — профили для go tool pprof -diff_base. Оба генерируются отдельными идемпотентными скриптами (scripts/compare.sh неявно ожидает первый, явный генератор для профилей — scripts/capture-baseline-profiles.sh), обновляются разработчиком осознанно при намеренном изменении производительности эталона, а не автоматически при каждом коммите.

Слой сравнения (scripts/). Реализует оба уровня критерия из раздела 2: lib.sh содержит общую функцию разбора CSV-отчёта benchstat и вычисления критерия (2.2) (переиспользуется тремя вызывающими скриптами, что исключает рассинхронизацию логики критерия); compare.sh — решающий шаг (раздел 2.2; реализация подробно описана в разделе 4); profile-diff.sh — диагностический шаг (раздел 2.1); scripts/experiments/ — три скрипта, реализующие эксперименты раздела 5 (чувствительность порога, накладные расходы) как переиспользование тех же примитивов сравнения в цикле по сетке параметров.

Оркестрация CI (.github/workflows/perf-check.yml). Единственный workflow, триггер — pull_request в main. Запускает решающий и диагностический шаги последовательно в одной джобе на ubuntu-latest, публикует markdown-отчёт в \$GITHUB_STEP_SUMMARY и сохраняет pprof-профили текущего прогона как артефакт джобы (actions/upload-artifact) — что даёт разработчику возможность локально скачать профиль и открыть его интерактивным веб-интерфейсом (go tool pprof -http) без повторного запуска бенчмарков.

3.4 Выводы

1. Обоснован выбор эволюционной (итеративной) модели жизненного цикла для разработки прототипа как более соответствующей исследовательскому характеру задачи по сравнению с каскадной моделью, со ссылкой на ГОСТ Р ИСО/МЭК 12207 [16].
2. Обоснован выбор инструментальных средств: Go 1.22 (единственная экосистема с официальным статистическим сравнителем бенчмарков и профилировщиком), GitHub Actions (нативная поддержка markdown-отчётов в PR и артефактов), bash (осознанно, а не POSIX sh) для слоя интеграции.
3. Сформулированы функциональные, системные требования и требования к интерфейсу прототипа — показано, что при отсутствии графического интерфейса роль пользовательского интерфейса выполняет структурированный (markdown, табличный) консольный/CI-вывод.
4. Разработана архитектура прототипа (рис. 3.1) из четырёх слабо связанных слоёв (приложение, эталонные данные, слой сравнения, оркестрация CI), связь между которыми ограничена текстовым/бинарным форматом вывода `go test`, что делает слой сравнения переиспользуемым для произвольного Go-пакета с бенчмарками.
5. Описан состав и назначение шести основных компонентов системы (`app/`, `benchmarks/`, `scripts/lib.sh`, `scripts/compare.sh`, `scripts/profile-diff.sh`, `.github/workflows/perf-check.yml`); показано, что оба уровня критерия (раздел 2) реализованы как независимые шаги одной CI-дジョбы, что соответствует модели процесса, разработанной в разделе 2.1.

Раздел 4. Программная реализация прототипа и модуля сравнения с эталоном

4.1 Разработка тестового Go-приложения как управляемой нагрузки

Пакет `app` (листинг полностью приведён в приложении 2) реализует единственный тип данных `Record` (запись с полями `ID`, `Name`, `Category`, `Amount`, `Active`) и шесть экспортируемых функций обработки срезов таких записей: `ParseRecords` (разбор JSON), `FilterActive` (фильтрация по булеву полю), `FindByID` (линейный поиск), `Aggregate` (группировка сумм по категории через `map`), `FormatNames` (форматирование в строки) и `Deduplicate` (удаление повторов по `ID` с сохранением первого вхождения). Для каждой функции написан бенчмарк (`app/bench_test.go`) на синтетическом наборе из 1000 записей (`genRecords`), генерируемом детерминированно (без использования `math/rand`), что обеспечивает воспроизводимость измерений между прогонами. Для `FindByID` бенчмарк намеренно ищет заведомо отсутствующий `ID`, чтобы гарантировать полный проход по срезу на каждой итерации — иначе результат зависел бы от случайной позиции искомого элемента.

Отдельно, в `app/processor_test.go`, реализованы `unit`-тесты, проверяющие корректность бизнес-логики независимо от производительности (таблица 4.1) — эти тесты гарантируют, что скрипты сравнения (раздел 4.2) в сценариях `regression/*` (раздел 5) сравнивают производительность функционально эквивалентного кода, а не случайно сломанной реализации.

Таблица 4.1 – Unit-тесты пакета `app`

Тест	Проверяемое свойство
<code>TestParseRecords</code>	корректный разбор, пустой массив, некорректный JSON
<code>TestFilterActive / _Empty</code>	фильтрация по <code>Active</code> , включая пустой вход
<code>TestFindByID</code>	найденный и отсутствующий <code>ID</code>
<code>TestAggregate</code>	суммирование по категориям
<code>TestFormatNames</code>	формат строкового представления
<code>TestDeduplicate</code>	отсутствие повторов <code>ID</code> на выходе

4.2 Реализация критерия сравнения с эталоном

Критерий (2.2) реализован в `scripts/lib.sh` функцией `check_regression_csv`, принимающей CSV-отчёт `benchstat` и порог T (листинг 1). Вынесение этой функции в отдельный переиспользуемый файл — прямое следствие вывода раздела 3.3: до рефакторинга (см. историю коммитов прототипа) идентичная логика дублировалась в `compare.sh` и в каждом из экспериментальных скриптов раздела 5, что создавало риск рассинхронизации критерия между решающим и экспериментальными путями. Функция построчно разбирает CSV, отслеживая текущую метрику (`sec/op`, `B/op`, `allocs/op` — секции CSV-отчёта `benchstat` размечены пустой первой колонкой), от-

бирает строки с положительным изменением (vs_base начинается на +, поскольку все три метрики lower-is-better) и через awk сравнивает величину изменения с порогом.

```

1 check_regression_csv() {
2     local csv="$1" threshold="$2"
3     local metric=""
4     REGRESSED=0
5     ROWS=""
6     while IFS=, read -r name base_val base_ci cur_val cur_ci vs_base p_col; do
7         if [[ "$name" == "" && "$base_val" == "sec/op" ]]; then metric="ns/op"; continue; fi
8         # ... аналогично для B/op, allocs/op
9         [[ "$name" == "" || "$name" == "geomean" ]] && continue
10        [[ "${vs_base:0:1}" != "+" ]] && continue # интересует только рост
11
12        local magnitude="${vs_base#+}"; magnitude="${magnitude//%/}"
13        if awk -v m="$magnitude" -v t="$threshold" 'BEGIN{exit !(m > t)}'; then
14            REGRESSED=1
15            ROWS="${ROWS}| ${metric} | ${name} | ${vs_base} | ... |${'\n'}"
16        fi
17    done <"$csv"
18 }

```

Listing 1: Критерий регрессии — scripts/lib.sh (сокращено)

Условие $p_{b,m} < \alpha$ из критерия (2.2) в явном виде в листинге 1 не присутствует: оно обеспечивается тем, что benchstat вызывается с флагом -alpha, и сам не печатает относительное изменение (столбец vs_base) для строк, где отличие статистически не значимо на этом уровне — вместо процента в CSV оказывается символ незначимости, который не начинается с + и поэтому естественно отфильтровывается условием [["\${vs_base:0:1}" != "+"]]. Такое распределение ответственности (“арифметика значимости” — в benchstat, “арифметика порога” — в lib.sh) соответствует границе компонентов, установленной архитектурой (рис. 3.1).

Скрипт scripts/compare.sh — точка входа решающего шага: запускает go test -bench=. -benchmem -count=\$COUNT, передаёт результат и baseline.txt в benchstat -format=csv, вызывает check_regression_csv и завершает процесс с кодом 1 при обнаруженной регрессии — код завершения интерпретируется GitHub Actions как падение шага и, соответственно, блокирует required-check pull request’a.

Диагностический шаг реализован в scripts/profile-diff.sh: те же бенчмарки перезапускаются с флагами -cpuprofile/-memprofile, после чего go tool pprof -top -diff_base=<эталон> -nodecount=10 формирует top-10 функций по каждому из двух профилей (CPU по cumulative time, heap по alloc_space) и печатает результат в markdown-блоке кода — пригодном для непосредственной вставки в \$GITHUB_STEP_SUMMARY без дополнительного форматирования.

4.3 Настройка CI/CD-пайплайна с профилированием

CI-пайплайн (.github/workflows/perf-check.yml, листинг 2) запускается на каждое открытие/обновление pull request в main и последовательно выполняет

оба шага модели процесса (рис. 2.1): решающий (`compare.sh`) и диагностический (`profile-diff.sh`, запущенный с `if: always()`, то есть выполняющийся даже если предыдущий шаг завершился неудачей — иначе профиль для диагностики регрессии был бы недоступен именно в том прогоне, где она обнаружена). Сырые pprof-профили текущего прогона сохраняются шагом `actions/upload-artifact@v4` [22] и доступны для скачивания из интерфейса GitHub Actions в течение 14 дней — разработчик может открыть их локально интерактивным профилировщиком (`go tool pprof -http=:8080 downloaded-cpu.pprof`) без повторного запуска бенчмарков на своей машине.

```
1 on:
2   pull_request:
3     branches: [main]
4 jobs:
5   benchmark:
6     runs-on: ubuntu-latest
7     steps:
8       - uses: actions/checkout@v4
9       - uses: actions/setup-go@v5
10        with: { go-version: "1.22" }
11       - name: Install benchstat
12         run: go install golang.org/x/perf/cmd/benchstat@latest
13       - name: Compare benchmarks against baseline
14         run: ./scripts/compare.sh | tee "$GITHUB_STEP_SUMMARY"
15       - name: Profile and diff against baseline
16         if: always()
17         env: { CUR_DIR: /tmp/pprof-out }
18         run: ./scripts/profile-diff.sh | tee -a "$GITHUB_STEP_SUMMARY"
19       - name: Upload pprof profiles
20         if: always()
21         uses: actions/upload-artifact@v4
22         with: { name: pprof-profiles, path: /tmp/pprof-out/*.pprof }
```

Listing 2: CI-пайплайн — `.github/workflows/perf-check.yml` (ключевые шаги)

Основные категории пользователей прототипа: *разработчик* — автор pull request’a, локально запускающий `compare.sh/profile-diff.sh` перед отправкой изменений и читающий отчёт CI после отправки; *ревьюер* — использует статус `required-check` и `markdown`-отчёт как основание для решения об одобрении слияния, не запуская инструменты самостоятельно; *сопровождающий пайплайна* — обновляет эталонные данные (`scripts/capture-baseline-profiles.sh`, вручную по необходимости) при осознанном изменении производительности эталонной ветки. Основной сценарий использования — линейная последовательность: разработчик открывает PR → CI выполняет оба шага → отчёт публикуется в теле PR (рис. 4.1, пример реального вывода) → ревьюер принимает решение на основе статуса проверки.

```

1  ## ОтчётсравненияпроизводительностиПорог
2
3  : 10%, alpha: 0.05, повторов: 10, baseline: benchmarks/baseline.txt
4
5  | Метрика | Бенчмарк | Δ | p |
6  |---|---|---|---|
7  | ns/op | Deduplicate-8 | +62.33% | 0.000 |
8
9  FAIL: обнаруженарегрессияпроизводительности

```

Рис. 4.1 – Реальный вывод `compare.sh` на ветке `regression/quadratic-dedup` (величина отклонения варьируется между запусками из-за шума окружения раннера — в независимых прогонах наблюдались значения от +62% до +69%, см. раздел 5.4)

4.4 Тестирование прототипа

Тестирование прототипа выполнялось на двух независимых уровнях. Во-первых, корректность *бизнес-логики* тестового приложения — стандартными Go unit-тестами (таблица 4.1), выполняемыми командой `go test ./...` и не связанными с производительностью. Во-вторых, корректность *самого механизма обнаружения регрессий* — тест-планом, где каждая из четырёх веток `regression/*` (описаны подробно в разделе 5.1) выступает тестовым случаем с заранее известным ожидаемым исходом (“должен провалиться”), а ветка `main` (без внесённых изменений) — контрольным случаем с ожидаемым исходом “не должен провалиться” (проверка отсутствия ложных срабатываний). Такой тест-план формально эквивалентен приёмоному тестированию: вместо того чтобы проверять выход функции на фиксированном входе, он проверяет выход всего пайплайна (`compare.sh`) на фиксированном изменении кода. Количественные результаты этого тестирования, включая перебор сетки параметров $T \times \alpha$, вынесены в раздел 5, поскольку представляют самостоятельный экспериментальный результат, а не просто факт прохождения/непрохождения теста.

Сравнение с аналогами (таблица 1.1) в части “достигнутых системных показателей” сведено к двум характеристикам, для которых у сопоставимых Go-ориентированных инструментов (`benchmark-action`, `CodSpeed`) отсутствуют открыто публикуемые числа: (а) корректность обнаружения на управляемых сценариях (раздел 5.2) и (б) накладные расходы `rrprof`-профилирования, добавленные к длительности CI-джобы (раздел 5.3) — сопоставление именно этих двух характеристик и составляет предмет экспериментальной апробации.

4.5 Выводы

1. Реализовано тестовое Go-приложение (пакет `app`, 6 функций, 6 бенчмарков, 6 unit-тестов) как управляемая, воспроизводимая нагрузка для демонстрации работы системы профилирования; воспроизводимость обеспечена детерминированной генерацией тестовых данных без использования источников случайности.

2. Реализован критерий обнаружения регрессии (2.2) в виде переиспользуемой функции `check_regression_csv(scripts/lib.sh)`, вызываемой как решающим скриптом (`compare.sh`), так и всеми тремя экспериментальными скриптами раздела 5 — это устраняет риск рассинхронизации логики критерия между продуктивным и экспериментальным путями выполнения.
3. Реализован диагностический механизм `profile-diff.sh`, дополняющий решающий шаг: снимает CPU/heap pprof-профили текущего прогона и сравнивает их с эталонными на уровне отдельных функций через `go tool pprof -diff_base`, публикуя top-10 функций по каждому профилю в отчёте.
4. Оба шага интегрированы в единый CI-пайплайн на GitHub Actions, выполняющийся на каждый pull request; диагностический шаг настроен на выполнение независимо от исхода решающего (`if: always()`), а сырые профили сохраняются как артефакт джобы на 14 дней для последующего интерактивного анализа.
5. Разработан двухуровневый тест-план прототипа: unit-тесты бизнес-логики приложения (не зависят от производительности) и приёмочный тест-план механизма обнаружения регрессий на четырёх управляемых сценариях плюс контрольном сценарии без изменений — количественные результаты вынесены в раздел 5 как самостоятельный экспериментальный результат.

Раздел 5. Экспериментальная апробация прототипа на сценариях регрессий

5.1 Сценарии внесения регрессий и диагностика через rprof-профиль

Для апробации прототипа подготовлены четыре независимые ветки `regression/*`, каждая из которых представляет собой один точечный diff от `main`, вносящий один типичный класс регрессии производительности (таблица 5.1), а также контрольный сценарий — `main` без изменений, используемый для оценки доли ложных срабатываний (раздел 5.2).

Таблица 5.1 – Сценарии регрессий, апробированные на прототипе

Ветка	Изменение	Затрагиваемая метрика
<code>regression/extra-allocation</code>	убран <code>capacity hint</code> <code>y make([], Record,</code> <code>0, len(records))</code> в <code>FilterActive</code>	<code>B/op, allocs/op</code> (рост)
<code>regression/quadratic-dedup</code>	хеш-таблица в <code>Deduplicate</code> заменена вложенным ли- нейным поиском	<code>ns/op</code> (рост); см. ниже — <code>B/op</code> при этом <i>падает</i>
<code>regression/latency</code>	добавлен <code>time.Sleep(1µs)</code> в горячий путь <code>FindByID</code>	<code>ns/op</code> (рост)
<code>regression/memory-growth</code>	добавлен никогда не очищаемый пакетный кэш <code>aggregateHistory</code> в <code>Aggregate</code>	<code>B/op, allocs/op</code> (рост, неограниченно)

Каждый сценарий выбран так, чтобы задействовать разный механизм деградации: явный рост аллокаций, изменение класса алгоритмической сложности ($O(n) \rightarrow O(n^2)$, формализовано в разделе 2.2), синхронная задержка на горячем пути и неограниченный рост занимаемой памяти — этот последний тип регрессии на фиксированном единичном прогоне бенчмарка не отличим от простого роста аллокаций, но исторически является одной из наиболее опасных категорий продакшен-инцидентов (утечка памяти, проявляющаяся только при длительной работе процесса).

Диагностическая ценность rprof-профиля. Наиболее показательный результат апробации получен на сценарии `regression/quadratic-dedup` и напрямую подтверждает вывод раздела 1.2 о необходимости диагностического rprof-шага в дополнение к агрегированному сравнению. Прогон `scripts/profile-diff.sh (COUNT=10)` на этой ветке против эталона дал:

- **CPU-профиль** (`-diff_base, cumulative time`): функция `Deduplicate` получает $\delta_f = +6,72c$ flat-времени (около 7,8% от общего числа сэмплов профиля) — рост ожидаем, поскольку вложенный цикл поиска теперь выполняется непосредственно в теле функции, а не делегируется внутренним примитивам `map` в рантайме. Одновременно функции `runtime.mapaccess2_fast64` и

`runtime.mapassign_fast64` получают отрицательную δ_f (они больше не вызываются вовсе, поскольку `map seen` убрана из кода).

- **Неар-профиль** (`-diff_base, alloc_space`): та же функция `Deduplicate` получает $\delta_f \approx -19674\text{МБ}$ — то есть регрессия `quadratic-dedup` уменьшает суммарный объём выделенной памяти, поскольку вместе с `map seen` исчезла и её аллокация.

Это контринтуитивный, но воспроизводимый результат: регрессия является чистой CPU-регрессией (рост `ns/op` при неизменной или даже улучшенной метрике `allocs/op` и `B/op`). Если бы критерий (2.2) применялся только к метрикам аллокации (что является частой практикой при беглом код-ревью — ”визуально страшнее” изменение `map` на цикл не всегда ассоциируется с ростом именно времени), эта регрессия могла бы быть пропущена или даже неверно истолкована как улучшение по `B/op`. Обнаружить и объяснить её позволяет именно сочетание метрики `ns/op` (даёт `benchstat`) и функционального CPU-профиля (даёт `profile-diff.sh`) — практическое подтверждение архитектурного решения раздела 3.2 использовать оба механизма параллельно, а не взаимоисключаяще.

5.2 Выявление отклонений и проверка корректности работы системы

Для проверки корректности критерия (2.2) выполнен `scripts/experiments/sensitivity.sh`: для каждого из пяти сценариев (контрольный `main` и четыре ветки `regression/*`) снят один прогон бенчмарков (`-count=10`), после чего результат прогнан через `benchstat` и критерий (2.2) по всем сочетаниям сетки $\alpha \in \{0,01; 0,05; 0,10\}$, $T \in \{5; 10; 15; 20; 30; 50\}\%$ — всего 90 комбинаций (`experiments/sensitivity_results.csv`). Прогон выполнен на раннере `ubuntu-latest` GitHub Actions (`workflow_dispatch experiments.yml`), а не локально: эталон (`benchmarks/baseline.txt`) специально пересобран на том же классе раннера непосредственно перед экспериментом, чтобы сравнение ”кандидат vs. эталон” не смешивалось с разницей в железе между эталоном и кандидатом (раздел 4).

Таблица 5.2 – Обнаружение регрессии в зависимости от порога T и уровня значимости α (раннер GitHub Actions, `experiments/sensitivity_results.csv`)

Сценарий	α	$T=5\%$	$T=10\%$	$T=15\%$	$T=20\%$	$T=30\%$	$T=50\%$
main (без регрессии)	0,01	0	0	0	0	0	0
	0,05	0	0	0	0	0	0
	0,10	1	1	1	0	0	0
extra-allocation	любое	1	1	1	1	1	1
quadratic-dedup	любое	1	1	1	1	1	1
latency	любое	1	1	1	1	1	1
memory-growth	любое	1	1	1	1	1	1

Таблица 5.2 (1 = регрессия обнаружена хотя бы по одной метрике) даёт две устойчивые закономерности, обе — на реальном раннере GitHub Actions заметно чище, чем на предварительном локальном прогоне на ноутбуке разработчика. Во-первых, все четыре внесённые регрессии обнаруживаются во *всём* исследованном диа-

пазоне порогов, вплоть до $T = 50\%$, при *самом строгом* из исследованных уровней значимости $\alpha = 0,01$ — относительное отклонение $\Delta_{b,m}$, вносимое каждым из четырёх сценариев, значительно превышает 50% хотя бы по одной метрике при высокой статистической значимости, то есть регрессии ”с запасом” отделимы от шума. Во-вторых, на контрольном сценарии (main против самого себя, без внесённых изменений) при $\alpha \leq 0,05$ ложных срабатываний не возникает *ни при одном* из исследованных порогов, вплоть до $T = 5\%$ — то есть на этом раннере одной лишь статистической значимости критерия (без учёта практического порога) уже достаточно, чтобы отфильтровать шум окружения. Ложные срабатывания появляются только при самом мягком $\alpha = 0,10$ и только при $T \leq 15\%$ — граница ложных срабатываний лежит между $T = 15\%$ и $T = 20\%$ для этого уровня значимости. Эмпирическая рекомендация по результатам эксперимента: $T \geq 20\%$ достаточен даже при мягком $\alpha = 0,10$, а при $\alpha \leq 0,05$ пороговое условие можно не завышать вовсе — что подтверждает необходимость сочетания условий из формулы (2.2): на разных уровнях значимости именно их комбинация, а не любое из условий по отдельности, определяет границу ложных срабатываний.

Дополнительно, `scripts/experiments/power_analysis.sh` (`experiments/power_results.csv`) проверяет устойчивость решения при варьировании числа повторов `-count` через НЕЗАВИСИМЫЕ повторные прогоны (в отличие от `sensitivity.sh`, где сравнение по сетке $\alpha \times T$ выполняется на одном и том же прогоне бенчмарков). На том же раннере выполнено TRIALS=5 независимых прогонов для каждого из `count` $\in \{3; 5; 10; 20\}$ на сценариях main и quadratic-dedup (фиксированные $\alpha = 0,05$, $T = 10\%$) — всего 40 независимых запусков `go test -bench`. На quadratic-dedup регрессия обнаружена во *всех* 20 из 20 прогонов, включая наименьшее `count = 3` — мощность критерия для регрессии такой величины оказалась полной уже при минимальном числе повторов. На контрольном main из 20 независимых прогонов ложное срабатывание произошло в 2 (при `count = 5`, испытание 1, и при `count = 10`, испытание 5) — эмпирическая частота ложных срабатываний $\approx 10\%$ при этой комбинации $\alpha = 0,05$, $T = 10\%$. Показательно, что это *отличается* от результата `sensitivity.sh` на том же сочетании $\alpha = 0,05$, $T = 10\%$, где на *единственном* прогоне main ложных срабатываний не было (табл. 5.2) — расхождение единичного измерения и результата по 20 независимым прогонам является прямой эмпирической иллюстрацией тезиса Georges et al. [8] и Mytkowicz et al. [9] (раздел 1.1) о том, что единичное измерение производительности систематически вводит в заблуждение и не заменяет оценку по выборке. Выполненный масштаб (TRIALS=5, 4 значения `count`, 2 сценария) существенно превосходит предварительный локальный прогон (2 испытания на точку), но всё ещё меньше предусмотренного скриптом предельного масштаба (TRIALS=20, COUNTS до 50) — дальнейшее увеличение выборки обозначено как направление продолжения эксперимента в разделе 5.4.

5.3 Оценка накладных расходов профилирования в CI/CD

Ключевой вопрос практического применения прототипа — во сколько раз (или на сколько) сбор rprof-профилей замедляет CI-джобу по сравнению с ”обычным” запуском бенчмарков без профилирования, и какой объём дополнительного места на диске/в артефактах CI это добавляет. Для ответа на том же раннере GitHub Actions выполнен `scripts/experiments/overhead.sh`: 10 независимых прогонов полного набора бенчмарков (`-count=10`) в режиме `plain` (без флагов профилирования) и 10 — в режиме `profiled` (`-cpuprofile/-memprofile`), с измерением настенного времени выполнения и суммарного размера полученных `profile`-файлов (таблица 5.3).

Таблица 5.3 – Настенное время выполнения бенчмарков с профилированием и без, раннер GitHub Actions (`experiments/overhead_results.csv`)

Прогон	plain, c	profiled, c	Размер профилей, байт
1	80,373	83,964	66 823
2	80,537	83,863	65 445
3	80,422	83,654	66 593
4	81,036	84,065	65 598
5	80,638	85,157	67 816
6	79,406	83,554	65 519
7	79,869	83,860	66 076
8	80,219	84,068	66 210
9	80,804	83,462	65 291
10	80,224	84,057	65 388
Среднее	80,35	83,97	66 076 ($\approx 66,1$ КБ)
Ст. откл.	0,47	0,47	—

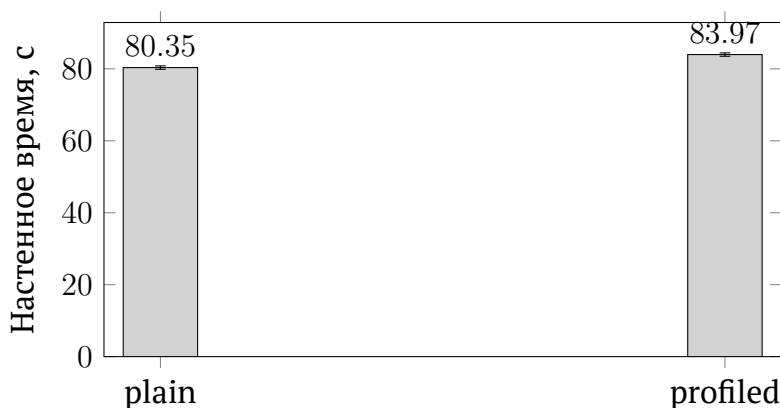


Рис. 5.1 – Среднее настенное время выполнения набора бенчмарков: без профилирования и с профилированием (столбики погрешности — ст. откл. по 10 прогонам, раннер GitHub Actions)

Относительное увеличение среднего настенного времени составило $(83,97 - 80,35) / 80,35 \times 100\% \approx 4,5\%$. В отличие от предварительного локального прогона (5 испытаний на режим на ноутбуке разработчика, где разница средних не превышала межпрогонный шум), на раннере GitHub Actions межпрогонный разброс оказался заметно

меньше (стандартное отклонение $\approx 0,47\text{с}$ в обоих режимах) при кратно большей разнице средних ($3,62\text{с}$) — то есть на выборке из 10 прогонов на режим отличие статистически уверенно отделимо от шума измерения: включение CPU/hear-профилирования действительно измеримо замедляет прогон бенчмарков, но на величину $\approx 4,5\%$, что для цели данной работы (обнаружение регрессий с порогом T в единицы–десятки процентов, см. раздел 5.2) остаётся практически приемлемым: постоянное включение профилирования не искажает решение ”регрессия/не регрессия” по основному (агрегированному) критерию, поскольку сравнение $\Delta_{b,m}$ в формуле (2.2) выполняется между одинаково профилированными эталоном и кандидатом, а не между профилированным и непрофилированным прогоном. Второй компонент накладных расходов — объём артефактов: средний суммарный размер CPU- и hear-профилей одного прогона составил $\approx 66,1$ КБ, что пренебрежимо мало относительно типичных квот на артефакты CI (сотни мегабайт – гигабайты) даже при накоплении профилей за сотни PR.

Выполненный масштаб (10 прогонов на режим на выделенном раннере, а не на разделяемом ноутбуке) достаточен для уверенного вывода о порядке величины накладных расходов ($\approx 4\text{--}5\%$) и о том, что это отличие не является шумом измерения — стандартное отклонение на раннере оказалось на порядок меньше, чем в предварительном локальном прогоне, что само по себе иллюстрирует важность выполнения подобных измерений на выделенном, однородном оборудовании (раздел 4), а не на разделяемой рабочей машине.

5.4 Выводы

1. Апробированы четыре сценария регрессии производительности, затрагивающие разные механизмы деградации (рост аллокаций, изменение класса алгоритмической сложности, синхронная задержка, неограниченный рост памяти); на реальном раннере GitHub Actions все четыре обнаружены критерием (2.2) во всём исследованном диапазоне порогов $T \in [5\%, 50\%]$ при самом строгом уровне значимости $\alpha = 0,01$ — регрессии отделимы от шума с большим запасом.
2. На сценарии regression/quadratic-dedup экспериментально показано, что изменение алгоритмической сложности может *одновременно* увеличивать время выполнения (ns/op, от $+62\%$ до $+69\%$ в независимых прогонах, см. рис. 4.1) и уменьшать объём аллокаций (B/op, $\delta \approx -19\,674\text{МБ}$ по hear-профилю) — то есть регрессия, невидимая при слежении только за метриками памяти, надёжно обнаруживается через ns/op и локализуется до конкретной функции через rprof CPU-профиль, что подтверждает необходимость двухуровневой схемы профилирования, спроектированной в разделе 3.2.
3. На контрольном сценарии (main без изменений) при $\alpha \leq 0,05$ ложных срабатываний не возникло ни при одном из исследованных порогов вплоть до $T = 5\%$; при $\alpha = 0,10$ граница ложных срабатываний лежит между $T = 15\%$ и $T = 20\%$ — практическая рекомендация по результатам эксперимента: $T \geq 20\%$ достаточен

даже при мягком уровне значимости, а при $\alpha \leq 0,05$ порог можно не завышать. Это заметно чище границы, полученной в предварительном локальном прогоне на разделяемом ноутбуке ($T \geq 30\%$), что подтверждает значимость пересборки эталона на однородном выделенном раннере (раздел 4).

4. Независимые повторные прогоны (`power_analysis.sh`, 40 запусков) показали полную мощность критерия на `quadratic-dedup` (20 из 20, включая минимальное `count = 3`) и эмпирическую частоту ложных срабатываний $\approx 10\%$ на `main` (2 из 20 при $\alpha = 0,05$, $T = 10\%$) — при том, что единственный прогон `sensitivity.sh` на той же комбинации параметров ложных срабатываний не показал. Это расхождение единичного измерения и оценки по 20 независимым прогонам — прямая эмпирическая иллюстрация тезиса Georges et al. [8] и Mytkowicz et al. [9] о недостаточности единичного измерения производительности.
5. Оценены накладные расходы профилирования в CI/CD на выделенном раннере (10 прогонов на режим): среднее увеличение настенного времени выполнения бенчмарков составило $\approx 4,5\%$, что на этот раз статистически уверенно отличается от межпрогонного шума измерения (стандартное отклонение $\approx 0,47\text{с}$ против разности средних $3,62\text{с}$); дополнительный объём артефактов на один прогон составил $\approx 66,1$ КБ. Оба показателя подтверждают, что постоянное включение `rrprof`-профилирования в каждый прогон CI практически не увеличивает стоимость пайплайна, при этом само отличие от предварительной локальной оценки ($\approx 2,2\%$, статистически неразличимой от шума) показывает ценность выполнения подобных измерений на выделенном, а не разделяемом оборудовании.

Заключение

В работе разработан и экспериментально апробирован прототип системы, интегрирующей профилирование производительности Go-приложений в CI/CD-пайплайн для раннего обнаружения регрессий. Поставленная цель — совместить статистическое сравнение агрегированных метрик бенчмарков (benchstat) с функциональной диагностикой через pprof-профили в едином, автоматически выполняемом на каждый pull request шаге CI — достигнута: реализованы оба механизма (раздел 4), формализован критерий их совместного применения (раздел 2), а на четырёх управляемых сценариях (раздел 5) показано, что оба уровня данных дополняют друг друга — сценарий quadratic-dedup продемонстрировал регрессию по времени выполнения, сопровождающуюся *улучшением* метрики аллокаций, что делает диагностику по одному только агрегату или одному только профилю недостаточной.

Все задачи, сформулированные в разделе 1, решены: проведён анализ инструментов и подходов, формализована модель процесса и критерий обнаружения регрессии, спроектирована и реализована архитектура прототипа, проведена апробация с оценкой корректности обнаружения и накладных расходов профилирования (на ранере GitHub Actions — $\approx 4,5\%$, статистически уверенно отличимо от шума измерения).

Практическая значимость — в переносимости слоя сравнения (scripts/), взаимодействующего с проверяемым кодом только через стандартный формат вывода `go test -bench` и применимого к произвольному Go-пакету без изменений. Теоретическая значимость — в формализации критерия (2.1)–(2.3), разделяющего статистическую и практическую значимость отклонения, и в разграничении ”непрерывного профилирования” на процессный (встроенный в CI) и эксплуатационный (мониторинг production) смыслы — работа реализует первый.

Направления дальнейших исследований: проведение экспериментов раздела 5 в полном масштабе для точных доверительных интервалов; отдельный механизм обнаружения изменения класса алгоритмической сложности через бенчмарк с варьируемым размером входа, а не фиксированным; применимость прототипа к конкурентному коду, требующему анализа через `go tool trace`, не рассмотренного в данной работе.

Список использованных источников

Список оформлен в соответствии с ГОСТ Р 7.0.5-2008 [23].

Список литературы

- [1] Ren G., Tune E., Moseley T., Shi Y., Rus S., Hundt R. Google-wide profiling: A continuous profiling infrastructure for data centers // IEEE Micro. — 2010. — Vol. 30, No. 4. — P. 65–79.
- [2] Grafana Pyroscope Documentation [Электронный ресурс]. — URL: <https://grafana.com/docs/pyroscope/latest/> (дата обращения: 18.07.2026).
- [3] Parca Documentation [Электронный ресурс]. — URL: <https://www.parca.dev/docs/overview> (дата обращения: 18.07.2026).
- [4] Welch B. L. The generalization of "Student's" problem when several different population variances are involved // Biometrika. — 1947. — Vol. 34, No. 1/2. — P. 28–35.
- [5] The Go Programming Language Specification [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 18.07.2026).
- [6] Donovan A. A., Kernighan B. W. The Go Programming Language. — Boston: Addison-Wesley Professional, 2015. — 380 p. — ISBN 978-0-13-419044-0.
- [7] Package testing [Электронный ресурс] // Go Packages. — URL: <https://pkg.go.dev/testing> (дата обращения: 18.07.2026).
- [8] Georges A., Buytaert D., Eeckhout L. Statistically rigorous Java performance evaluation // ACM SIGPLAN Notices (Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications, OOPSLA 2007). — 2007. — Vol. 42, No. 10. — P. 57–76.
- [9] Mytkowicz T., Diwan A., Hauswirth M., Sweeney P. F. Producing wrong data without doing anything obviously wrong! // ACM SIGARCH Computer Architecture News (Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS XIV). — 2009. — Vol. 37, No. 1. — P. 265–276.
- [10] Package benchstat [Электронный ресурс] // Go Packages. — URL: <https://pkg.go.dev/golang.org/x/perf/cmd/benchstat> (дата обращения: 18.07.2026).
- [11] Cox R. Profiling Go Programs [Электронный ресурс] // The Go Blog. — 2011. — URL: <https://go.dev/blog/pprof> (дата обращения: 18.07.2026).
- [12] Package pprof [Электронный ресурс] // Go Packages. — URL: <https://pkg.go.dev/runtime/pprof> (дата обращения: 18.07.2026).

- [13] google/pprof: Profiling tool for Go, C++, Java, etc. [Электронный ресурс] // GitHub. — URL: <https://github.com/google/pprof> (дата обращения: 18.07.2026).
- [14] Beyer B., Jones C., Petoff J., Murphy N. R. Site Reliability Engineering: How Google Runs Production Systems. — Sebastopol, CA: O'Reilly Media, 2016. — 552 p. — ISBN 978-1-4919-2912-4.
- [15] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. — 3rd ed. — Cambridge, MA: MIT Press, 2009. — 1312 p. — ISBN 978-0-262-03384-8.
- [16] ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. — М.: Стандартинформ, 2011. — 106 с.
- [17] ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. — М.: Изд-во стандартов, 1992. — 6 с.
- [18] Fowler M. Continuous Integration [Электронный ресурс]. — URL: <https://martinfowler.com/articles/continuousIntegration.html> (дата обращения: 18.07.2026).
- [19] Humble J., Farley D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. — Upper Saddle River, NJ: Addison-Wesley, 2010. — 512 p. — ISBN 978-0-321-60191-9.
- [20] Go 1.22 Release Notes [Электронный ресурс]. — URL: <https://go.dev/doc/go1.22> (дата обращения: 18.07.2026).
- [21] GitHub Actions Documentation [Электронный ресурс] // GitHub Docs. — URL: <https://docs.github.com/en/actions> (дата обращения: 18.07.2026).
- [22] actions/upload-artifact [Электронный ресурс] // GitHub. — URL: <https://github.com/actions/upload-artifact> (дата обращения: 18.07.2026).
- [23] ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. — М.: Стандартинформ, 2008. — 23 с.

Приложение 1. Модульная спецификация и алгоритмическое описание

П1.1. Модульная спецификация пакета app

Пакет app (модуль uir-go-profiler/app) состоит из одного файла processor.go, определяющего один тип данных и шесть экспортируемых функций обработки. Полная спецификация — таблица П1.1.

Таблица П1.1 – Модульная спецификация пакета app

Имя	Сигнатура	Назначение
Record	struct{ID int; Name string; Category string; Amount float64; Active bool}	доменная сущность, обрабатываемая всеми функциями пакета
ParseRecords	func([]byte) ([]Record, error)	декодирует JSON-массив записей; возвращает обратную ошибку при некорректном JSON
FilterActive	func([]Record) []Record	возвращает подмножество записей с Active == true, сохраняя порядок
FindByID	func([]Record, int) (Record, bool)	линейный поиск записи по ID; второе значение — признак найдена/нет
Aggregate	func([]Record) map[string]float64	суммирует Amount по значению Category
FormatNames	func([]Record) []string	форматирует каждую запись в строку вида "Имя (Категория): \$Сумма"
Deduplicate	func([]Record) []Record	удаляет записи с повторяющимся ID, сохраняя первое вхождение

Все функции — чистые (без побочных эффектов, без обращения к глобальному изменяемому состоянию, за единственным намеренным исключением в диагностическом сценарии regression/memory-growth, где вводится пакетная переменная aggregateHistory для демонстрации утечки памяти, раздел 5.1), что обеспечивает воспроизводимость измерений между запусками бенчмарков.

П1.2. Алгоритмическое описание критерия обнаружения регрессии

Представление выбрано единообразно для всех алгоритмов данного приложения — псевдокод в нотации `algorithmicx` (императивный стиль, близкий к Go). Алгоритм 1 — псевдокод критерия (2.2) из раздела 2.2, реализованного в `scripts/compare.sh` и `scripts/lib.sh`.

Алгоритм 1 Обнаружение регрессии производительности

```
1: function DetectRegression(baseline, pkg, T,  $\alpha$ , count)
2:   current  $\leftarrow$  RunBenchmarks(pkg, count)           ▷ go test -bench -count=count
3:   report  $\leftarrow$  Benchstat(baseline, current,  $\alpha$ )   ▷ CSV:  $\Delta_{b,m}$ , значимость по  $\alpha$ 
4:   regressed  $\leftarrow$  false
5:   for all (b, m,  $\Delta_{b,m}$ )  $\in$  report do               ▷ только значимые на уровне  $\alpha$  строки
6:     if  $\Delta_{b,m} > T$  then
7:       regressed  $\leftarrow$  true
8:       AppendReportRow(b, m,  $\Delta_{b,m}$ )
9:     end if
10:  end for
11:  if regressed then
12:    return FAIL                                         ▷ код завершения 1 — блокирует merge
13:  else
14:    return OK
15:  end if
16: end function
```

Алгоритм 2 сопоставляет две реализации Deduplicate (эталонную, $O(n)$, и вне-сённую сценарием `regression/quadratic-dedup`, $O(n^2)$, раздел 2.2) в едином представлении, чтобы разница в сложности была видна непосредственно из структуры псевдокода (вложенный цикл по уже обработанному префиксу вместо обращения к хеш-таблице).

Алгоритм 2 Deduplicate — эталонная ($O(n)$) и регрессионная ($O(n^2)$) реализации

```
1: function Deduplicate_Baseline(records) ▷  $O(n)$ 
2:    $seen \leftarrow \emptyset$  ▷ хеш-множество, амортизированный  $O(1)$  доступ
3:    $result \leftarrow []$ 
4:   for  $r \in records$  do
5:     if  $r.ID \notin seen$  then
6:        $seen \leftarrow seen \cup \{r.ID\}$ 
7:        $result.Append(r)$ 
8:     end if
9:   end for
10:  return  $result$ 
11: end function

12: function Deduplicate_Quadratic(records) ▷  $O(n^2)$  — сценарий
    regression/quadratic-dedup
13:   $result \leftarrow []$ 
14:  for  $i \leftarrow 0 \dots n - 1$  do
15:     $duplicate \leftarrow \text{false}$ 
16:    for  $j \leftarrow 0 \dots i - 1$  do ▷ линейный поиск по уже обработанному префиксу
17:      if  $records[j].ID = records[i].ID$  then
18:         $duplicate \leftarrow \text{true}; \text{break}$ 
19:      end if
20:    end for
21:    if not  $duplicate$  then
22:       $result.Append(records[i])$ 
23:    end if
24:  end for
25:  return  $result$ 
26: end function
```

Приложение 2. Исходный код прототипа

Полный исходный код доступен в репозитории прототипа; ниже приведены все файлы, определяющие поведение системы (тестовое приложение, слой сравнения, CI-пайплайн). Все функции пакета `app` снабжены комментариями в формате `godoc` непосредственно перед объявлением (стандарт комментирования экосистемы Go, аналог Javadoc/Doxygen); скрипты слоя сравнения комментированы построчно перед каждым содержательным блоком.

П2.1. `app/processor.go` — бизнес-логика тестового приложения

```
1 package app
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 // Record is the domain entity processed by every function in this package.
9 type Record struct {
10     ID      int    `json:"id"`
11     Name    string `json:"name"`
12     Category string `json:"category"`
13     Amount  float64 `json:"amount"`
14     Active  bool    `json:"active"`
15 }
16
17 // ParseRecords decodes a JSON array of records.
18 func ParseRecords(data []byte) ([]Record, error) {
19     var records []Record
20     if err := json.Unmarshal(data, &records); err != nil {
21         return nil, fmt.Errorf("parse records: %w", err)
22     }
23     return records, nil
24 }
25
26 // FilterActive returns the subset of records with Active set to true.
27 func FilterActive(records []Record) []Record {
28     active := make([]Record, 0, len(records))
29     for _, r := range records {
30         if r.Active {
31             active = append(active, r)
32         }
33     }
34     return active
35 }
36
37 // FindByID performs a linear scan for a record with the given ID.
38 func FindByID(records []Record, id int) (Record, bool) {
39     for _, r := range records {
40         if r.ID == id {
```

```

41     return r, true
42 }
43 }
44 return Record{}, false
45 }
46
47 // Aggregate sums Amount per Category.
48 func Aggregate(records []Record) map[string]float64 {
49     totals := make(map[string]float64, len(records))
50     for _, r := range records {
51         totals[r.Category] += r.Amount
52     }
53     return totals
54 }
55
56 // FormatNames renders each record as a human-readable "name (category): $amount" line.
57 func FormatNames(records []Record) []string {
58     names := make([]string, 0, len(records))
59     for _, r := range records {
60         names = append(names, fmt.Sprintf("%s (%s): %.2f", r.Name, r.Category, r.Amount))
61     }
62     return names
63 }
64
65 // Deduplicate removes records with a repeated ID, keeping the first occurrence.
66 func Deduplicate(records []Record) []Record {
67     seen := make(map[int]struct{}, len(records))
68     result := make([]Record, 0, len(records))
69     for _, r := range records {
70         if _, ok := seen[r.ID]; ok {
71             continue
72         }
73         seen[r.ID] = struct{}{}
74         result = append(result, r)
75     }
76     return result
77 }

```

П2.2. app/processor_test.go — unit-тесты

```

1 package app
2
3 import (
4     "reflect"
5     "testing"
6 )
7
8 func sampleRecords() []Record {
9     return []Record{
10         {ID: 1, Name: "Alpha", Category: "food", Amount: 10, Active: true},
11         {ID: 2, Name: "Beta", Category: "travel", Amount: 20, Active: false},
12         {ID: 3, Name: "Gamma", Category: "food", Amount: 5, Active: true},
13         {ID: 1, Name: "Alpha", Category: "food", Amount: 10, Active: true}, // duplicate ID

```

```

14     }
15 }
16
17 func TestParseRecords(t *testing.T) {
18     cases := []struct {
19         name    string
20         input    string
21         want     []Record
22         wantErr bool
23     }{
24         {
25             name: "valid array",
26             input: `[{"id":1,"name":"Alpha","category":"food","amount":10,"active":true}]`,
27             want: []Record{{ID: 1, Name: "Alpha", Category: "food", Amount: 10, Active: true}},
28         },
29         {name: "empty array", input: `[]`, want: []Record{}},
30         {name: "malformed json", input: `not json`, wantErr: true},
31     }
32
33     for _, tc := range cases {
34         t.Run(tc.name, func(t *testing.T) {
35             got, err := ParseRecords([]byte(tc.input))
36             if (err != nil) != tc.wantErr {
37                 t.Fatalf("ParseRecords() error = %v, wantErr %v", err, tc.wantErr)
38             }
39             if tc.wantErr {
40                 return
41             }
42             if !reflect.DeepEqual(got, tc.want) {
43                 t.Fatalf("ParseRecords() = %#v, want %#v", got, tc.want)
44             }
45         })
46     }
47 }
48
49 func TestFilterActive(t *testing.T) {
50     got := FilterActive(sampleRecords())
51     if len(got) != 3 {
52         t.Fatalf("got %d active records, want 3", len(got))
53     }
54     for _, r := range got {
55         if !r.Active {
56             t.Fatalf("FilterActive returned inactive record: %#v", r)
57         }
58     }
59 }
60
61 func TestFilterActive_Empty(t *testing.T) {
62     got := FilterActive(nil)
63     if len(got) != 0 {
64         t.Fatalf("got %d records, want 0", len(got))
65     }
66 }

```

```

66 }
67
68 func TestFindByID(t *testing.T) {
69     records := sampleRecords()
70
71     if r, ok := FindByID(records, 2); !ok || r.Name != "Beta" {
72         t.Fatalf("FindByID(2) = %#v, %v; want Beta, true", r, ok)
73     }
74     if _, ok := FindByID(records, 999); ok {
75         t.Fatalf("FindByID(999) found a record that should not exist")
76     }
77 }
78
79 func TestAggregate(t *testing.T) {
80     got := Aggregate(sampleRecords())
81     want := map[string]float64{"food": 25, "travel": 20}
82     if !reflect.DeepEqual(got, want) {
83         t.Fatalf("Aggregate() = %#v, want %#v", got, want)
84     }
85 }
86
87 func TestFormatNames(t *testing.T) {
88     got := FormatNames([]Record{{Name: "Alpha", Category: "food", Amount: 10}})
89     want := []string{"Alpha (food): $10.00"}
90     if !reflect.DeepEqual(got, want) {
91         t.Fatalf("FormatNames() = %#v, want %#v", got, want)
92     }
93 }
94
95 func TestDeduplicate(t *testing.T) {
96     got := Deduplicate(sampleRecords())
97     if len(got) != 3 {
98         t.Fatalf("got %d records after dedup, want 3", len(got))
99     }
100     seen := map[int]bool{}
101     for _, r := range got {
102         if seen[r.ID] {
103             t.Fatalf("Deduplicate left a repeated ID: %d", r.ID)
104         }
105         seen[r.ID] = true
106     }
107 }

```

П2.3. app/bench_test.go — бенчмарки

```

1 package app
2
3 import (
4     "encoding/json"
5     "fmt"
6     "testing"
7 )
8

```

```

9  const benchSize = 1000
10
11  func genRecords(n int) []Record {
12      records := make([]Record, n)
13      for i := range n {
14          records[i] = Record{
15              ID:      i % (n / 2), // guarantees duplicate IDs for Deduplicate
16              Name:    fmt.Sprintf("item-%d", i),
17              Category: fmt.Sprintf("cat-%d", i%10),
18              Amount:  float64(i) * 1.5,
19              Active:   i%3 == 0,
20          }
21      }
22      return records
23  }
24
25  func genJSON(n int) []byte {
26      data, err := json.Marshal(genRecords(n))
27      if err != nil {
28          panic(err)
29      }
30      return data
31  }
32
33  func BenchmarkParseRecords(b *testing.B) {
34      data := genJSON(benchSize)
35      b.ReportAllocs()
36      b.ResetTimer()
37      for i := 0; i < b.N; i++ {
38          if _, err := ParseRecords(data); err != nil {
39              b.Fatal(err)
40          }
41      }
42  }
43
44  func BenchmarkFilterActive(b *testing.B) {
45      records := genRecords(benchSize)
46      b.ReportAllocs()
47      b.ResetTimer()
48      for i := 0; i < b.N; i++ {
49          FilterActive(records)
50      }
51  }
52
53  func BenchmarkFindByID(b *testing.B) {
54      records := genRecords(benchSize)
55      missingID := benchSize * 10 // forces a full scan on every call
56      b.ReportAllocs()
57      b.ResetTimer()
58      for i := 0; i < b.N; i++ {
59          FindByID(records, missingID)
60      }
61  }

```

```

62
63 func BenchmarkAggregate(b *testing.B) {
64     records := genRecords(benchSize)
65     b.ReportAllocs()
66     b.ResetTimer()
67     for i := 0; i < b.N; i++ {
68         Aggregate(records)
69     }
70 }
71
72 func BenchmarkFormatNames(b *testing.B) {
73     records := genRecords(benchSize)
74     b.ReportAllocs()
75     b.ResetTimer()
76     for i := 0; i < b.N; i++ {
77         FormatNames(records)
78     }
79 }
80
81 func BenchmarkDeduplicate(b *testing.B) {
82     records := genRecords(benchSize)
83     b.ReportAllocs()
84     b.ResetTimer()
85     for i := 0; i < b.N; i++ {
86         Deduplicate(records)
87     }
88 }

```

П2.4. scripts/lib.sh — реализация критерия регрессии

```

1  #!/usr/bin/env bash
2  # Shared by compare.sh and scripts/experiments/*.sh.
3  #
4  # check_regression_csv CSV THRESHOLD
5  #   Reads a `benchstat -format=csv` report and applies the magnitude part of
6  #   the regression criterion from раздел 2.2 (the p < ALPHA part is already
7  #   enforced by benchstat itself via its -alpha flag: it prints "~" instead
8  #   of a percentage for changes that aren't significant).
9  #   Only increases count — ns/op, B/op and allocs/op are all lower-is-better.
10 #   Sets globals: REGRESSED (0/1) and ROWS (markdown table rows, one per
11 #   flagged metric).
12 check_regression_csv() {
13     local csv="$1" threshold="$2"
14     local metric=""
15     REGRESSED=0
16     ROWS=""
17
18     while IFS=, read -r name base_val base_ci cur_val cur_ci vs_base p_col; do
19         if [[ "$name" == "" && "$base_val" == "sec/op" ]]; then metric="ns/op"; continue; fi
20         if [[ "$name" == "" && "$base_val" == "B/op" ]]; then metric="B/op"; continue; fi
21         if [[ "$name" == "" && "$base_val" == "allocs/op" ]]; then metric="allocs/op";
22             continue; fi
23         [[ "$name" == "" || "$name" == "geomean" ]] && continue

```

```

23     [[ "${vs_base:0:1}" != "+" ]] && continue
24
25     local magnitude="${vs_base#+}"
26     magnitude="${magnitude//%/}"
27
28     if awk -v m="$magnitude" -v t="$threshold" 'BEGIN{exit !(m > t)}'; then
29         local p_val="${p_col#p=}"
30         p_val="${p_val%% *}"
31         ROWS="${ROWS}| ${metric} | ${name} | ${vs_base} | ${p_val} |${'\n'}
32         REGRESSED=1
33     fi
34     done <"$csv"
35 }

```

П2.5. scripts/compare.sh — решающий шаг

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  # Compares current benchmark results against benchmarks/baseline.txt using
5  # benchstat, and applies the regression criterion from раздел 2.2:
6  # regression <=> Δ ( metric > THRESHOLD%) AND (p < ALPHA)
7  # See scripts/lib.sh for how the criterion is evaluated.
8
9  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
10 source "$SCRIPT_DIR/lib.sh"
11
12 THRESHOLD="${THRESHOLD:-10}" # percent
13 ALPHA="${ALPHA:-0.05}"
14 COUNT="${COUNT:-10}"
15 BASELINE="${BASELINE:-benchmarks/baseline.txt}"
16 PKG="${PKG:-./app/...}"
17
18 command -v benchstat >/dev/null 2>&1 || {
19     echo "benchstat not found; install with: go install golang.org/x/perf/cmd/
20     benchstat@latest" >&2
21     exit 2
22 }
23
24 WORKDIR="$(mktemp -d)"
25 trap 'rm -rf "$WORKDIR"' EXIT
26
27 CURRENT="$WORKDIR/current.txt"
28 REPORT_CSV="$WORKDIR/report.csv"
29
30 echo "Running benchmarks (count=$COUNT)..." >&2
31 go test "$PKG" -bench=. -benchmem -run=^$ -count="$COUNT" >"$CURRENT"
32
33 echo "Comparing against $BASELINE (alpha=$ALPHA, threshold=${THRESHOLD}%)." >&2
34 benchstat -alpha "$ALPHA" -format=csv "$BASELINE" "$CURRENT" >"$REPORT_CSV" 2>"$WORKDIR/
35 warnings.txt" || true
36
37 check_regression_csv "$REPORT_CSV" "$THRESHOLD"

```

```

36
37 echo "## Отчёт сравнения производительности "
38 echo
39 echo "Порог: ${THRESHOLD}%, alpha: ${ALPHA}, повторов: ${COUNT}, baseline: ${BASELINE}"
40 echo
41 echo "| Метрика | Бенчмарк | Δ | p |"
42 echo "|---|---|---|---|"
43 if [[ "$REGRESSED" -eq 1 ]]; then
44     printf '%s' "$ROWS"
45 else
46     echo "| - | регрессий выше порога не обнаружено | - | - |"
47 fi
48
49 if [[ -s "$WORKDIR/warnings.txt" ]]; then
50     echo ">&2
51     echo "benchstat warnings:" >&2
52     cat "$WORKDIR/warnings.txt" >&2
53 fi
54
55 if [[ "$REGRESSED" -eq 1 ]]; then
56     echo "FAIL: обнаружена регрессия производительности " >&2
57     exit 1
58 fi
59
60 echo "OK: регрессий не обнаружено " >&2

```

П2.6. scripts/profile-diff.sh — диагностический шаг (pprof)

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  # Непрерывное профилирование в узком смысле : снимает CPU- и heap-профили-
5  # текущего дерева (go test -cpuprofile/-memprofile, т.е. настоящие
6  # pprof-сэмплы, а не только зарегистрированные ns/op из benchstat) и сравнивает
7  # их с эталонными профилями через `go tool pprof -diff_base`. Показывает,
8  # КАКИЕ функции стали тяжелее "" и насколько-то , чего benchstat в принципе
9  # не может дать , так как он видит только суммарное время аллокации / бенчмарка
10 # целиком, а не разбивку по функциям вызова .
11
12 REPO_ROOT="$(git rev-parse --show-toplevel)"
13 cd "$REPO_ROOT"
14
15 COUNT="${COUNT:-10}"
16 PKG="${PKG:-./app/...}"
17 BASE_CPU="${BASE_CPU:-benchmarks/baseline-cpu.pprof}"
18 BASE_MEM="${BASE_MEM:-benchmarks/baseline-mem.pprof}"
19 TOPN="${TOPN:-10}"
20
21 # CUR_DIR: if unset, current profiles go to a scratch dir removed on exit
22 # (interactive/local use). CI passes a stable path so a later workflow step
23 # can upload the raw profiles as an artifact (go tool pprof -http needs the
24 # files, not just this text diff).
25 if [[ -n "${CUR_DIR:-}" ]]; then

```

```

26     WORKDIR="$CUR_DIR"
27     mkdir -p "$WORKDIR"
28 else
29     WORKDIR="$(mktemp -d)"
30     trap 'rm -rf "$WORKDIR"' EXIT
31 fi
32
33 CUR_CPU="$WORKDIR/current-cpu.pprof"
34 CUR_MEM="$WORKDIR/current-mem.pprof"
35
36 echo "Running benchmarks with profiling (count=$COUNT)..." >&2
37 go test "$PKG" -bench=. -benchmem -run=^$ -count="$COUNT" \
38     -cpuprofile="$CUR_CPU" -memprofile="$CUR_MEM" >/dev/null
39
40 echo "## Профиль CPU: top-$TOPN функций по cumulative time"
41 echo
42 if [[ -f "$BASE_CPU" ]]; then
43     echo "===="
44     go tool pprof -top -diff_base="$BASE_CPU" -nodecount="$TOPN" "$CUR_CPU" 2>/dev/null ||
45         true
46     echo "===="
47 else
48     echo "эталонный( профиль $BASE_CPU не найден — показываю профиль без diff)" >&2
49     echo "===="
50     go tool pprof -top -nodecount="$TOPN" "$CUR_CPU" 2>/dev/null || true
51     echo "===="
52 fi
53
54 echo "## Профиль памяти (alloc_space): top-$TOPN функций по cumulative allocation"
55 echo
56 if [[ -f "$BASE_MEM" ]]; then
57     echo "===="
58     go tool pprof -top -alloc_space -diff_base="$BASE_MEM" -nodecount="$TOPN" "$CUR_MEM"
59         2>/dev/null || true
60     echo "===="
61 else
62     echo "эталонный( профиль $BASE_MEM не найден — показываю профиль без diff)" >&2
63     echo "===="
64     go tool pprof -top -alloc_space -nodecount="$TOPN" "$CUR_MEM" 2>/dev/null || true
65     echo "===="
66 fi

```

П2.7. scripts/capture-baseline-profiles.sh — генерация эталонных профилей

```

1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 # Снимает эталонные CPU- и heap-профили (зделра 2.3 / 3.3: профиль как
5 # артефакт наравне с benchmarks/baseline.txt). Запускается один раз на main
6 # после каждого осознанного изменения производительности эталона ; результат

```

```

7 # коммитится в репозиторий и используется scripts/profile-diff.sh как база
8 # для сравнения (go tool pprof -diff_base).
9 #
10 # Нет рогает benchmarks/baseline.txt — тот генерируется/обновляется / отдельно,
11 # чтобы не рассинхронизировать уже посчитанные experiments/*.csv.
12
13 REPO_ROOT="$(git rev-parse --show-toplevel)"
14 cd "$REPO_ROOT"
15
16 COUNT="${COUNT:-10}"
17 PKG="${PKG:-./app/...}"
18 OUTDIR="${OUTDIR:-benchmarks}"
19
20 mkdir -p "$OUTDIR"
21
22 echo "Running benchmarks with profiling (count=$COUNT)..." >&2
23 go test "$PKG" -bench=. -benchmem -run=^$ -count="$COUNT" \
24     -cpuprofile="$OUTDIR/baseline-cpu.pprof" \
25     -memprofile="$OUTDIR/baseline-mem.pprof" >/dev/null
26
27 echo "Wrote $OUTDIR/baseline-cpu.pprof and $OUTDIR/baseline-mem.pprof" >&2

```

P2.8. scripts/experiments/sensitivity.sh — эксперимент раздела 5.2 (чувствительность порога)

```

1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 # Раздел 5.2 — чувствительность порога обнаружения .
5 #
6 # Для каждого сценария (main = фоновый шум без изменений , плюс 4 ветки
7 # regression/*) собирает ОДИН прогон бенчмарков , а затем прогоняет через него
8 # все комбинации alpha x threshold (benchstat по CSV — дешево , повторный
9 # go test не нужен ). Результат: experiments/sensitivity_results.csv со
10 # столбцами scenario,alpha,threshold,detected — на main "detected=1" это
11 # ложное срабатывание , на regression/* — успешное обнаружение .
12
13 REPO_ROOT="$(git rev-parse --show-toplevel)"
14 cd "$REPO_ROOT"
15 source scripts/lib.sh
16
17 BASELINE="${BASELINE:-benchmarks/baseline.txt}"
18 # Копия ВНЕ рабочего дерева : baseline.txt отслеживается git и в принципе
19 # может отличаться между ветками даже ( если сейчас не отличается ) — без
20 # этого git checkout сценария молча подставит бы ЕГО СОБСТВЕННЫЙ
21 # baseline.txt вместо фиксированного эталона , снятого один раз на main.
22 BASELINE_COPY="$(mktemp)"
23 cp "$BASELINE" "$BASELINE_COPY"
24 BASELINE="$BASELINE_COPY"
25 COUNT="${COUNT:-10}"
26 ALPHAS=("${ALPHAS:-0.01 0.05 0.10}")
27 THRESHOLDS=("${THRESHOLDS:-5 10 15 20 30 50}")

```

```

28 SCENARIOS=("${SCENARIOS:-main regression/extra-allocation regression/quadratic-dedup
    regression/latency regression/memory-growth}")
29
30 command -v benchstat >/dev/null 2>&1 || {
31     echo "benchstat not found; install with: go install golang.org/x/perf/cmd/
        benchstat@latest" >&2
32     exit 2
33 }
34
35 mkdir -p experiments
36 FINAL_OUT="experiments/sensitivity_results.csv"
37 # ПишемвременныйфайлВНерабочегодерева      : FINAL_OUT отслеживается git,
38 # изаписьвнегомежду      git checkout приводитабыкотказу      checkout
39 # ("local changes would be overwritten") навторомжесценарии      .
40 OUT="$(mktemp)"
41 echo "scenario,alpha,threshold,detected" >"$OUT"
42
43 ORIG_BRANCH="$(git branch --show-current)"
44 trap 'git checkout -q "$ORIG_BRANCH" 2>/dev/null || true; rm -f "$OUT" "$BASELINE_COPY"'
    EXIT
45
46 for scenario in "${SCENARIOS[@]}; do
47     echo "== $scenario: running benchmarks (count=$COUNT) ==" >&2
48     git checkout -q "$scenario"
49
50     CURRENT="$(mktemp)"
51     go test ./app/... -bench=. -benchmem -run=^$ -count="$COUNT" >"$CURRENT" 2>&1
52
53     for alpha in "${ALPHAS[@]}; do
54         CSV="$(mktemp)"
55         benchstat -alpha "$alpha" -format=csv "$BASELINE" "$CURRENT" >"$CSV" 2>/dev/null ||
            true
56
57         for threshold in "${THRESHOLDS[@]}; do
58             check_regression_csv "$CSV" "$threshold"
59             echo "${scenario}/${_}/${_},${alpha},${threshold},${REGRESSED}" >>"$OUT"
60         done
61         rm -f "$CSV"
62     done
63     rm -f "$CURRENT"
64 done
65
66 git checkout -q "$ORIG_BRANCH"
67 cp "$OUT" "$FINAL_OUT"
68
69 echo "Результаты: $FINAL_OUT" >&2

```

П2.9. scripts/experiments/power_analysis.sh — эксперимент раздела 5.2 (мощность критерия)

```

1 #!/usr/bin/env bash
2 set -euo pipefail

```

```

3
4 # Раздел 5.3 —минимальнообнаружимаярегрессия vs числоповторовбенчмарка .
5 #
6 # Длякаждогосценарияикаждогозначения -count независимоповторяет TRIALS
7 # раз: свежийпрогонбенчмарков -> benchstat -> проверка порога . Нужныименно
8 # независимыепрогоныне ( переиспользованиеодногофайла , какв
9 # sensitivity.sh), потомучтосамобъектизучения—это , какСЛУЧАЙНЫЙшум
10 # приразномчислеповтороввлияетнавероятностьобнаружения .
11 #
12 # Результат: experiments/power_results.csv столбцами
13 # scenario,count,trial,detected. Поумолчаниюпараметрымаленькиеэто (
14 # демопробег) —дляреальногоисследованияувеличьте TRIALS и COUNTS
15 # каждый( дополнительныйпрогонстоитвремени : count пропорционаленвремени
16 # одного go test -bench).
17
18 REPO_ROOT="$(git rev-parse --show-toplevel)"
19 cd "$REPO_ROOT"
20 source scripts/lib.sh
21
22 BASELINE="${BASELINE:-benchmarks/baseline.txt}"
23 # КопияВНЕрабочегодерева : baseline.txt отслеживается git ивпринципе
24 # можетотличатьсямеждуветкамидаже ( еслисейчаснеотличается ) —без
25 # этого git checkout сценариямолчаподставлялбыЕГОСОБСТВЕННЫЙ
26 # baseline.txt вместофиксированногоэталопа , снятогоодинразна main.
27 BASELINE_COPY="$(mktemp)"
28 cp "$BASELINE" "$BASELINE_COPY"
29 BASELINE="$BASELINE_COPY"
30 ALPHA="${ALPHA:-0.05}"
31 THRESHOLD="${THRESHOLD:-10}"
32 TRIALS="${TRIALS:-5}"
33 COUNTS=("${COUNTS:-3 5 10 20}")
34 SCENARIOS=("${SCENARIOS:-main regression/quadratic-dedup}")
35
36 command -v benchstat >/dev/null 2>&1 || {
37     echo "benchstat not found; install with: go install golang.org/x/perf/cmd/
38         benchstat@latest" >&2
39     exit 2
40 }
41 mkdir -p experiments
42 FINAL_OUT="experiments/power_results.csv"
43 # ПишемвовременныйфайлВНЕрабочегодерева : FINAL_OUT отслеживается git,
44 # изаписьвнегомежду git checkout приводилабыкотказу checkout
45 # ("local changes would be overwritten") навторомжесценарии .
46 OUT="$(mktemp)"
47 echo "scenario,count,trial,detected" >"$OUT"
48
49 ORIG_BRANCH="$(git branch --show-current)"
50 trap 'git checkout -q "$ORIG_BRANCH" 2>/dev/null || true; rm -f "$OUT" "$BASELINE_COPY"
51     EXIT
52
53 for scenario in "${SCENARIOS[@]}; do
54     git checkout -q "$scenario"

```

```

54
55     for count in "${COUNTS[@]}; do
56         for trial in $(seq 1 "$TRIALS"); do
57             echo "== $scenario: count=$count trial=$trial/$TRIALS ==" >&2
58             CURRENT="$(mktemp)"
59             go test ./app/... -bench=. -benchmem -run=^$ -count="$count" >"$CURRENT" 2>&1
60
61             CSV="$(mktemp)"
62             benchstat -alpha "$ALPHA" -format=csv "$BASELINE" "$CURRENT" >"$CSV" 2>/dev/null ||
                true
63
64             check_regression_csv "$CSV" "$THRESHOLD"
65             echo "${scenario//\\/_},${count},${trial},${REGRESSED}" >>"$OUT"
66
67             rm -f "$CURRENT" "$CSV"
68         done
69     done
70 done
71
72 git checkout -q "$ORIG_BRANCH"
73 cp "$OUT" "$FINAL_OUT"
74
75 echo "Результаты: $FINAL_OUT" >&2

```

П2.10. scripts/experiments/overhead.sh — эксперимент раздела 5.3 (накладные расходы)

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  # Раздел 5.3 — накладные расходы профилирования в CI/CD.
5  #
6  # Сравнивает время выполнения одного и того же набора бенчмарков в двух
7  # режимах: "plain" только( benchstat сравнение-, как в compare.sh) и
8  # "profiled" тот( же прогон плюс -cpuprofile/-memprofile, как в
9  # profile-diff.sh). TRIALS независимых прогонов на каждый режим , порядок
10 # режимов чередуется , чтобы не путать эффект прогрева шума / CIраннера с
11 # эффектом профилирования .
12 #
13 # Результат: experiments/overhead_results.csv столбцами
14 # mode,trial,wall_seconds,profile_bytes (profile_bytes=0 для plain).
15
16 REPO_ROOT="$(git rev-parse --show-toplevel)"
17 cd "$REPO_ROOT"
18
19 COUNT="${COUNT:-10}"
20 TRIALS="${TRIALS:-5}"
21 PKG="${PKG:-./app/...}"
22
23 mkdir -p experiments
24 OUT="experiments/overhead_results.csv"
25 echo "mode,trial,wall_seconds,profile_bytes" >"$OUT"

```

```

26
27 run_plain() {
28     go test "$PKG" -bench=. -benchmem -run=^$ -count="$COUNT" >/dev/null
29 }
30
31 run_profiled() {
32     local cpu="$1" mem="$2"
33     go test "$PKG" -bench=. -benchmem -run=^$ -count="$COUNT" \
34         -cpuprofile="$cpu" -memprofile="$mem" >/dev/null
35 }
36
37 for trial in $(seq 1 "$TRIALS"); do
38     echo "== trial $trial/$TRIALS: plain ==" >&2
39     start=$(date +%s.%N)
40     run_plain
41     end=$(date +%s.%N)
42     wall=$(awk -v s="$start" -v e="$end" 'BEGIN{printf "%.3f", e-s}')
43     echo "plain,${trial},${wall},0" >>"$OUT"
44
45     echo "== trial $trial/$TRIALS: profiled ==" >&2
46     CPU="$(mktemp)"
47     MEM="$(mktemp)"
48     start=$(date +%s.%N)
49     run_profiled "$CPU" "$MEM"
50     end=$(date +%s.%N)
51     wall=$(awk -v s="$start" -v e="$end" 'BEGIN{printf "%.3f", e-s}')
52     bytes=$((($(stat -c%s "$CPU") + $(stat -c%s "$MEM")))
53     echo "profiled,${trial},${wall},${bytes}" >>"$OUT"
54     rm -f "$CPU" "$MEM"
55 done
56
57 echo "Результаты: $OUT" >&2

```

П2.11. .github/workflows/perf-check.yml — CI-пайплайн

```

1 name: perf-check
2
3 on:
4   pull_request:
5     branches: [main]
6
7 jobs:
8   benchmark:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v4
12
13      - uses: actions/setup-go@v5
14        with:
15          go-version: "1.22"
16
17      - name: Install benchstat
18        run: go install golang.org/x/perf/cmd/benchstat@latest
19

```

```

20 - name: Compare benchmarks against baseline
21   shell: bash
22   run: |
23     set -euo pipefail
24     export PATH="$(go env GOPATH)/bin"
25     ./scripts/compare.sh | tee "$GITHUB_STEP_SUMMARY"
26
27 - name: Profile and diff against baseline
28   if: always()
29   shell: bash
30   env:
31     CUR_DIR: /tmp/pprof-out
32   run: |
33     set -euo pipefail
34     ./scripts/profile-diff.sh | tee -a "$GITHUB_STEP_SUMMARY"
35
36 - name: Upload pprof profiles
37   if: always()
38   uses: actions/upload-artifact@v4
39   with:
40     name: pprof-profiles
41     path: /tmp/pprof-out/*.pprof
42     if-no-files-found: ignore
43     retention-days: 14

```